



Klassificeringsprestanda för modellerna Naiv Bayes, logistisk regression och stödvektormaskin vid en begränsad datamängd

Classification Performance of the Models Naive Bayes, Logistic Regression, and Support Vector Machine with a Limited Data Set

Kandidatarbete inom civilingenjörsutbildningen vid Chalmers

Ronja Andersson

Linda Gao

Madeleine Larsson

Liam Lundfeldt

Klassificeringsprestanda för modellerna Naiv
Bayes, logistisk regression och stödvektormaskin
vid en begränsad datamängd

*Kandidatarbete i matematik inom civilingenjörsprogrammet Automation och meka-
tronik vid Chalmers*

Ronja Andersson Liam Lundfeldt

*Kandidatarbete i matematik inom civilingenjörsprogrammet Informationsteknik vid
Chalmers*

Madeleine Larsson

*Kandidatarbete i matematik inom civilingenjörsprogrammet Teknisk matematik vid
Chalmers*

Linda Gao

Handledare: Maria Roginskaya Institutionen för Matematiska vetenskaper
 Paula Femenias Institutionen för Arkitektur och samhällsbyggnadsteknik

Institutionen för Matematiska vetenskaper
CHALMERS TEKNISKA HÖGSKOLA
GÖTEBORGS UNIVERSITET
Göteborg, Sverige 2026

Förord

Först och främst vill vi rikta ett stort tack till vår handledare Maria Roginskaya och till vår bihandledare Paula Femenias för värdefull vägledning och stöd genom hela arbetet.

Under arbetets gång har en veckovis loggbok förts över de individuella prestationerna. Denna kompletterades med en dagbok som sammanfattade gruppens aktiviteter under veckan. Nedan följer en redogörelse för arbetsfördelningen inom gruppen och de individuella bidragen till föreliggande kandidatarbete. Samtliga författare har haft ett gemensamt ansvar för arbetets övergripande utformning, granskningen av rapportens innehåll samt för genomförandet av den inledande litteraturstudien och forskningen kring tidigare arbeten inom området.

Ronja Andersson har ansvarat för konstruktion och format av de filer som krävts för att koppla data till rätt kategori. Vidare har Ronja haft ansvaret för planerings- och slutrapportens strukturella uppbyggnad, samt genomfört träning och finjustering av modellen stödvektormaskin (SVM) för att uppnå bästa möjliga prestanda.

Linda Gao har ansvarat för utveckling och implementering av samtliga klassificeringsmodeller. Vidare har hon ansvarat för modellutvärdering och resultatanalys, med särskilt fokus på Naiv Bayes och logistisk regression. Hon har även haft ett övergripande ansvar för rapportens tekniska och matematiska innehåll.

Madeleine Larsson har ansvarat för att studien som helhet hänger ihop, med avseende på såväl innehåll som struktur, men inte med avseende på den tekniska implementationen. Specifikt har hon ansvarat för formulering av syfte och delproblem samt för att studien korresponderar mot dessa. Speciellt har hon också ansvarat för att just slutrapporten hänger ihop som helhet, exempelvis med avseende på begreppen generalisering och överanpassning, vilket är ett genomgående tema. Madeleine har också ansvarat för grundläggande teori om maskininlärning, för teori kopplad till problemformulering och för teori kopplad till modellen SVM. Samt för att studien har viss grund i vetenskapliga källor. Hon har också skapat mallar för hur projektet kan följas upp, tillsammans med Ronja ansvarat för samhälleliga och etiska aspekter samt haft en guidande roll gällande källhänvisningar.

Liam Lundfeldt har ansvarat för datahantering och transformation av rådata till JSON-format. Arbetet var fokuserat på undersökning och utvärdering av olika metodiker för datapreparering, där Liam har implementerat de förbehandlingssteg som krävts för att optimera modellernas slutgiltiga prestanda. Liam har också ansvarat för kommunikation och mötesplanering med handledare.

På nästa sida följer en tabell över vem eller vilka som haft huvudsakligt ansvar för de olika avsnitten i rapporten.

Nr	Avsnitt	Huvudförfattare
	Sammandrag	Larsson
	Abstract	Lundfeldt
	Förord	Andersson, Gao Larsson & Lundfeldt
	1 Inledning	
1.1	Bakgrund	Larsson
1.2	Syfte	Larsson
1.3	Delproblem	Larsson
1.4	Avgränsningar	Andersson & Larsson
1.5	Samhälleliga och etiska aspekter	Andersson & Larsson
	2 Teori	
2.1	Begränsade datamängder	Larsson
2.1.1	Problemet med begränsade datamängder	Larsson
2.1.2	Hantering av problemet med överanpassning	Larsson
2.2	Låg datakvalitet	Larsson
2.2.1	Exempel på låg datakvalitet	Larsson
2.2.2	Hantering av låg datakvalitet	Larsson
2.3	Förbehandling av textdata	Lundfeldt
2.3.1	Agglomerativ klustring	Lundfeldt
2.4	Textrepresentation	Gao
2.4.1	Påse med ord (BoW)	Gao
2.4.2	Termfrekvenser-invers dokumentfrekvenser (TF-IDF)	Gao
2.4.3	N-grams	Andersson
2.5	Klassificeringsmodeller	Gao
2.5.1	Naiv Bayes	Gao
2.5.2	Logistisk regression	Gao
2.5.3	Stödvektormaskin (SVM)	Larsson
2.6	Prestandamått	Andersson
2.7	Träning och testning	Lundfeldt
2.7.1	Sampling	Lundfeldt
2.7.2	K-faldig korsvalidering	Gao
	3 Implementation	
3.1	Beskrivning av data	Andersson & Larsson
3.2	Förbehandling av data	Andersson & Lundfeldt
3.3	Urval av tränings- och testmängder	Andersson & Gao
3.4	Modellträning och testning med olika textrepresentationer	Andersson
3.5	Prestandamått	Andersson
	4 Resultat och diskussion	
4.1	Urvalsstrategier för träningsdata	Gao
4.2	Naiv Bayes	Gao
4.3	Logistisk regression	Gao
4.4	Stödvektormaskin	Andersson
4.5	Övergripande analys inklusive generaliseringsförmåga	Gao
	5 Slutsatser	Andersson, Gao & Larsson

Tabell 1: Ansvarsfördelning för rapportens huvudavsnitt.

Nr	Avsnitt	Huvudförfattare
Appendix		
A	Användning av AI	Gao
B	Teori: Maskininlärning	Larsson
C	Teori: Studie av prestanda vid begränsade datamängder	Larsson
D	Teori: Prestandamått	Andersson
E	Resultat och diskussion: Val av prestandamått	Andersson
F	Resultat och diskussion: Hyperparametrar med högst makro F1-poäng	Gao
G	Resultat och diskussion: Modellsensitivitet	Lundfeldt
H	Resultat och diskussion: Modellöverensstämmelse	Lundfeldt
I	Resultat och diskussion: Ord med stor påverkan	Lundfeldt
J	Källkod	Lundfeldt & Gao
J.1	Textextrahering och förbehandling	Lundfeldt
J.1.1	Klustring och skapande av synonymmappning	Lundfeldt
J.1.2	Synonymomvandling	Lundfeldt
J.2	Modellernas kärnkod	Gao
J.2.1	Naiv Bayes	Gao
J.2.2	Logistisk regression	Gao
J.2.3	SVM	Gao
J.2.4	Gemensamma hjälpfunktioner	Gao

Tabell 2: Ansvarsfördelning för rapportens appendix.

Sammandrag

Den här studien rör sig inom ämnesområdet maskininlärning. Syftet har övergripande varit att bidra med kunskap i situationer där det är önskvärt att ta fram modeller för klassificering av text, men då mängden träningsdata är liten. Denna kunskap bedöms vara värdefull för organisationer som vill klassificera sin egen organisationsspecifika information. Problemet med begränsade datamängder vid maskininlärning är ett erkänt problem inom vetenskapen och de modeller som tränas upp på sådana datamängder kan få bristande generaliseringsförmåga. Mer specifikt har syftet med studien varit att undersöka och jämföra prestandan för tre olika typer av modeller för textklassificering: Naiv Bayes, logistisk regression och stödvektormaskin (SVM). Detta vid träning på en begränsad mängd träningsdata och vid binär klassificering.

I studien har 448 intervjuer utgjort datamängden. Denna data var etiketterad, vilket möjliggjorde övervakad maskininlärning, men av låg kvalitet. Datamängden kan beskrivas som högdimensionell, obalanserad och brusig. Inför träning och testning av klassificeringsmodellerna så förbehandlades data, olika urval av träningsmängder undersöktes och olika prestandamått utvärderades. Det visade sig vara lämpligt att använda balanserad fördelning av träningsdata, med lika många intervjuer från varje klass, och prestandamåttet makro F1-poäng. Under träning och testning, så modifierades modellparametrar och textrepresentation. Två textrepresentationsmetoder användes: Påse med ord (Bag of Words, BoW) och TF-IDF. Vid testning användes metoden k-faldig korsvalidering (k-fold cross-validation).

Samtliga modeller uppnådde som bäst en makro F1-poäng på cirka 0,6 då de testades på testdata. Eftersom prestandan var lägre på testdata än på träningsdata indikerar detta överanpassning och låg generaliseringsförmåga. Urval av träningsmängd och textrepresentationsmetod hade stor påverkan på resultatet.

Den övergripande slutsatsen är att ingen av klassificeringsmodellerna Naiv Bayes, logistisk regression eller SVM uppnår hög prestanda då de har tränats på denna studies begränsade datamängd. En bidragande orsak till låg prestation var dock bristande datakvalitet, vars påverkan skulle kunna undersökas vidare.

Abstract

This study is conducted in the field of machine learning and aims to contribute knowledge regarding the use of machine learning in text classification in situations where only limited amounts of training data are available. Such knowledge is assessed valuable for organizations that seek to classify organization-specific information. The problem of limited training data is a recognized challenge in machine learning, as models trained on such datasets can have limited generalization capabilities. More specifically, the study investigates and compares the performance of three different text classification models, namely Naive Bayes, Logistic Regression and Support Vector Machine (SVM). This was examined under conditions of limited training data and binary classification.

The dataset used in this study consisted of 448 labeled interviews, which enabled supervised machine learning. Furthermore the data exhibited characteristics of high dimensionality, imbalance and noise. Before training and testing the classification models, the data was preprocessed, different training set configurations were examined, and various performance metrics were evaluated. Balanced training distributions, with an equal number of interviews from each class, and the performance metric F1-score were found to be suitable. During training and testing, model parameters and text representations were modified. Two text representation methods were used, Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF). During testing the method k-fold cross-validation was used.

The results show that all models achieved at best a F1-score of approximately 0.6 on test data. Because the performance was lower on test data compared to training data, the results indicate overfitting, and low generalization capabilities. The selection of training data and the choice of text representation method were found to have a substantial impact on performance.

In conclusion, none of the aforementioned classification models achieved high performance when trained on the limited dataset examined in this study. However, one contributing factor to the relatively low performance was deemed to be the limited quality of the available data, a factor that could be further investigated.

Innehåll

1	Inledning	1
1.1	Bakgrund	1
1.2	Syfte	1
1.3	Delproblem	1
1.4	Avgränsningar	2
1.5	Samhälleliga och etiska aspekter	2
2	Teori	3
2.1	Begränsade datamängder	3
2.1.1	Problemet med begränsade datamängder	3
2.1.2	Hantering av problemet med överanpassning	4
2.2	Låg datakvalitet	4
2.2.1	Exempel på låg datakvalitet	4
2.2.2	Hantering av låg datakvalitet	5
2.3	Förbehandling av textdata	5
2.3.1	Agglomerativ klustering	5
2.4	Textrepresentation	6
2.4.1	Påse med ord (BoW)	6
2.4.2	Termfrekvenser-invers dokumentfrekvenser (TF-IDF)	6
2.4.3	N-grams	6
2.5	Klassificeringsmodeller	7
2.5.1	Naiv Bayes	7
2.5.2	Logistisk regression	8
2.5.3	Stödvektormaskin (SVM)	9
2.6	Prestandamått	11
2.7	Träning och testning	11
2.7.1	Sampling	11
2.7.2	K-faldig korsvalidering	11
3	Implementation	12
3.1	Beskrivning av data	12
3.2	Förbehandling av data	12
3.3	Urval av tränings- och testmängder	13
3.4	Modellträning och testning med olika textrepresentationer	13
3.5	Prestandamått	14
4	Resultat och diskussion	14
4.1	Urvalsstrategier för träningsdata	14
4.2	Naiv Bayes	15
4.3	Logistisk regression	16
4.4	Stödvektormaskin	17
4.5	Övergripande analys inklusive generaliseringsförmåga	18
5	Slutsatser	20
	Källförteckning	21
	Appendix	i
A	Användning av AI	i
B	Teori: Maskininlärning	i
C	Teori: Studie av prestanda vid begränsade datamängder	iii

D Teori: Prestandamått	iv
D.1 Noggrannhet	iv
D.2 Precision	iv
D.3 Känslighet/Klassvis noggrannhet	iv
D.4 F1-poäng	v
E Resultat och diskussion: Val av prestandamått	vi
F Resultat och diskussion: Hyperparametrar med högst makro F1-poäng	viii
G Resultat och diskussion: Modellsensitivitet	ix
H Resultat och diskussion: Modellöverensstämmelse	xi
I Resultat och diskussion: Ord med stor påverkan	xiii
J Källkod	xv
J.1 Textextrahering och förbehandling	xv
J.1.1 Klustering och skapande av synonymmappning	xvi
J.1.2 Synonymomvandling	xvii
J.2 Modellernas kärnkod	xviii
J.2.1 Naiv Bayes	xviii
J.2.2 Logistisk regression	xxiii
J.2.3 SVM	xxvii
J.2.4 Gemensamma hjälpfunktioner	xxxii

1 Inledning

I detta kapitel ges en övergripande beskrivning av studien. Dess bakgrund och vad den syftar till behandlas, men det redogörs också för de konkreta delproblem som har adresserats och för vilka avgränsningar som har gjorts. Kapitlet innehåller också en beskrivning av de samhälleliga och etiska aspekter som studien har tagit hänsyn till.

1.1 Bakgrund

Studien rör sig inom ämnesområdet maskininlärning. Maskininlärning handlar enkelt uttryckt om att träna modeller på data för att de sedan ska kunna göra förutsägelser om nya situationer med ny data (Deisenroth m. fl., 2020, s. 4). I samband med den teknologiska utvecklingen har en allt större datamängd blivit tillgänglig, till exempel data från klimatsensorer, olika transaktioner och sociala media, något som ibland kallas ”big data” (Chiera & Korolkiewicz, 2017, s. 1). Generellt gäller att en större datamängd att träna en modell på ger bättre förutsägelser (Kinoshita & Mizuno, 2017, s. 93).

Men många organisationer som inte är stora teknikföretag har inte tillgång till stora datamängder och de kan ha behov av specifika applikationer (Gutman & Goldmeier, 2021, s. 168–169). En intressant frågeställning är då om de kan ta fram egna applikationer genom träning på sin begränsade mängd data och hur bra dessa skulle prestera. Att det kan vara problematiskt att träna modeller på begränsade datamängder är ett erkänt problem inom vetenskapen och det finns teori som beskriver problemet. Det har dessutom blivit fler forskningspublikationer i det här ämnet (Kokol m. fl., 2022).

Men även om det finns teori som beskriver problematiken kring begränsade datamängder, så tycks det inte finnas lika många fallbeskrivningar av hur bra eller dåligt modeller som har tränats på begränsade datamängder faktiskt presterar i olika situationer och under olika förutsättningar. Något som troligtvis är intressant att få en uppfattning om ifall man överväger att ta fram en egen modell och ett område som den här studien har inriktat sig på att bidra till. En specifik sådan situation skulle kunna vara när man har en begränsad mängd textdata som man önskar klassificera – ett fall som det inte är långsökt att tänka sig att många organisationer ställs inför.

1.2 Syfte

Syftet med studien är övergripande att bidra med kunskap i situationer där det är önskvärt att använda maskininlärning för att ta fram modeller för klassificering av text, men där mängden träningsdata att träna modellerna på är liten.

Syftet med studien har mer specifikt varit att undersöka och jämföra prestandan för tre olika typer av modeller för textklassificering: Naiv Bayes, logistisk regression och stödvektormaskin (SVM). Prestandan har undersökts då modellerna för textklassificering har tränats på en begränsad mängd träningsdata och för binär klassificering. Prestandan har också undersökts under olika dataförbehandlingssteg, textrepresentationer och urval av träningsmängd.

1.3 Delproblem

Syftet har brutits ned i följande delproblem:

1. Hur bör modellernas prestanda mätas?
2. Hur påverkar olika dataförbehandlingssteg prestandan hos modellerna?
3. Hur påverkar olika textrepresentationer prestandan hos modellerna?
4. Hur påverkar olika urval av träningsmängd prestandan hos modellerna?
5. Vilken prestanda kan modellerna Naiv Bayes, logistisk regression och SVM uppnå på en begränsad mängd träningsdata då klassificering ska ske i binära klasser?
6. Hur presterar de framtagna modellerna jämfört med varandra?

1.4 Avgränsningar

För att säkerställa ett resultat av hög kvalitet inom tidsramen har ett antal avgränsningar gjorts. För det första har studien fokuserat på övervakad maskininlärning, eftersom den data studien tillhandahölls var försedd med etiketter som möjliggjorde denna typ av maskininlärning. För mer information om övervakad maskininlärning, se Appendix B.

För det andra har studien fokuserat endast på tre olika typer av modeller: Naiv Bayes, logistisk regression och SVM. Dessa modeller är intressanta att undersöka därför att samtliga används vid övervakad maskininlärning, men samtidigt utgör exempel på olika typer av matematiska modeller. Naiv Bayes och logistisk regression är probabilistiska modeller och SVM handlar om att separera vektorer med ett hyperplan. För mer information om olika modeller, se Appendix B och avsnitt 2.5.

Studien har också avgränsat sig till enklare former av dataförbehandlingssteg. Det har ansetts vara ett lämpligt första steg och mest intressant för personer och organisationer utan stora resurser.

Vidare har arbetet avgränsats till att använda textrepresentationsmetoderna Påse med ord (Bag of Words, BoW) och Termfrekvenser-inversa dokumentfrekvenser (Term Frequency-Inverse Document Frequency, TF-IDF) för att omvandla texterna till ett format som modellerna kan avläsa.

Eftersom syftet med studien just är att undersöka en situation med begränsad mängd träningsdata, har det inte ansetts vara intressant att på olika sätt öka mängden träningsdata. Datamängden har bestått av 448 intervjuer och studien har fokuserat på modellernas tekniska prestanda och inte på att genomföra en analys av och dra slutsatser från de intervjuer som utgör datamängden.

1.5 Samhälleliga och etiska aspekter

Ur ett samhällsperspektiv förväntas studien, vilket framkom i avsnitt 1.2, bidra med kunskap i situationer där det är önskvärt att använda maskininlärning för att ta fram modeller för klassificering av text, men där mängden träningsdata att träna modellerna på är liten. Framförallt bedöms denna kunskap vara värdefull för organisationer som vill klassificera sin egen organisationsspecifika information, men som inte har stora datamängder att träna en modell på. Sådana organisationer kan med hjälp av denna rapport både förstå hur man praktiskt kan gå tillväga för att träna en egen modell på egen data och få en uppfattning om vilka prestandanivåer som är möjliga, även om förutsättningarna kan skilja sig åt i olika fall. Ambitionen ur ett samhälleligt perspektiv har också varit att vara så pedagogiska som möjligt för att det ska vara enkelt att ta till sig kunskapen.

Ur ett akademiskt perspektiv och till forskare förväntas studien bidra med en praktisk fallbeskrivning som visar vilken prestanda olika typer av modeller faktiskt uppnått vid begränsad mängd träningsdata under vissa premisser.

Det är generellt viktigt att vara medveten om att de klassificeringar som görs av en modell som tränats med hjälp av maskininlärning beror av den data den tränats på. Om träningsdata inte är representativ för verkligheten kan olika typer av skador uppkomma på grund av felklassificeringar. Denna problematik med att resultatet från modellen avviker från verkligheten kan benämnas partiskhet (bias) (Lindholm m. fl., 2022, s. 80). Detta är ett problem som är inneboende i själva metoden att använda träningsdata för att träna en modell och är därför inte ett problem som kan undvikas helt. Det har inte heller varit praktiskt möjligt att påverka dataunderlaget inom ramen för studiens begränsade tid. Istället har ambitionen varit att denna rapport ska vara pedagogiskt skriven så att det tydligt framkommer att den data som används som träningsdata spelar roll för den framtagna modellens funktionssätt.

I studien har istället integritetsaspekten och hanteringen av data varit den viktigaste etiska aspekten att ta hänsyn till. Den data som modellerna har tränats på är intervjudata, producerad inom ramen för Chalmers verksamhet, och denna data har behövt hanteras respektfullt. Denna data hade redan genomgått anonymisering då projektgruppen fick ta del av den, men ytterligare metadata rensades bort i början av studien, så att endast intervjusvar och dess kategorisering återstod. Data

laddades heller inte upp på någon extern samarbetsyta som inte godkänts av Chalmers.

2 Teori

I detta kapitel beskrivs den teori som har legat till grund för studien. Kapitlet inleds med ett avsnitt om problemet med begränsade datamängder vid maskininlärning. Därefter följer ett avsnitt om problem relaterade till låg datakvalitet. Resten av kapitlet behandlar olika aspekter som praktiskt behöver beaktas vid maskininlärning.

För den som önskar en mer grundläggande bakgrundsbeskrivning av maskininlärning återfinns detta i Appendix B, denna beskrivning inkluderar även ett förtydligande av vad som avses med ”modell” i denna studie. I Appendix C presenteras även en vetenskaplig studie som har undersökt vilken prestanda modeller kan uppnå trots begränsad mängd träningsdata. Denna studie är inte direkt jämförbar med detta kandidatarbets studie, men kan ändå fungera som en referenspunkt.

2.1 Begränsade datamängder

Som redan framkommit, kan ett problem vid framtagande av modeller genom maskininlärning vara att den mängd träningsdata som finns tillgänglig att lära från är begränsad. Detta problem och hur det övergripande kan hanteras beskrivs i detta avsnitt.

En relevant första frågeställning är dock hur liten en begränsad datamängd egentligen är. Enligt Naser (2026), finns det inte någon generell regel för vad som kan betraktas som en begränsad datamängd. Det finns dock varierande riktlinjer, till exempel att det ska vara minst tio exempel för varje variabel för linjära modeller för att överanpassning, se avsnitt 2.1.1, ska kunna undvikas.

2.1.1 Problemet med begränsade datamängder

Som framkom i avsnitt 1.1, så är problemet med begränsade datamängder vid träning av maskininlärningsmodeller ett erkänt problem inom vetenskapen. Kokol m.fl. (2022) har gått igenom forskningspublikationer som relaterar till både maskininlärning och begränsade datamängder och har påvisat ett ökande antal sådana publikationer. Antalet publikationer började stiga år 2003 och började stiga kraftigt år 2016.

Naser (2026) har undersökt utmaningar inom maskininlärning kopplade till begränsade datamängder och ett grundläggande problem som författaren beskriver är att begränsade datamängder gör det svårt att träna upp en modell med hög generaliseringsförmåga. Orsaken uppges vara den potentiellt stora risken för att modellerna i verkligheten kommer att tvingas behandla data som inte är representerad i träningsdatas distribution. Begränsad träningsdata täcker med andra ord sannolikt inte alla möjliga fall och då kan modellen prestera dåligt i verkligheten.

Lindholm m.fl. (2022, s. 65) menar dock att målet inom maskininlärning just är att uppnå så god prestation som möjligt på ny data som modellen inte tränats på. När de behandlar ämnet generaliseringsförmåga gör de det i termer av generaliseringsgap, vilket handlar om hur väl en modell presterar på ny data som modellen inte tidigare sett jämfört med hur väl modellen presterar på den data som den tränats på (Lindholm m.fl., 2022, s. 72). Ju mindre skillnad mellan dessa prestationer, desto mindre generaliseringsgap. Vilket kan tolkas som en högre generaliseringsförmåga. De konfirmerar också det resonemang som Naser (2026) för och menar att man kan förvänta sig att mer träningsdata leder till ett mindre generaliseringsgap (Lindholm m.fl., 2022, s. 75) (d.v.s. högre generaliseringsförmåga), men de menar också att mer träningsdata generellt leder till bättre prestation på ny data (Lindholm m.fl., 2022, s. 76).

Både fallet då en modell anpassar sig i hög grad till träningsdata och fallet då en modell anpassar sig i låg grad till träningsdata kan leda till dålig prestation på ny data (Lindholm m.fl., 2022, s. 73). Om anpassningen är för hög kallas det överanpassning (overfitting) och om anpassningen är för låg kallas det underanpassning (underfitting). Målet är att hitta en balans. Naser

(2026) tar upp just överanpassning som ett problem som relaterar till begränsade datamängder och menar att det eventuellt också är det mest allmängiltiga problemet i sammanhanget. Ett exempel på vad som praktiskt skulle kunna hända och som Naser (2026) tar upp är att parametrarna för den modell som tränas anpassas efter ett specifikt extremvärde, något vi menar är ett exempel på överanpassning. Det är svårare att hitta litteratur som berör ämnet begränsade datamängder och underanpassning, men datorföretaget Lenovo (u.å.) skriver att en otillräcklig mängd träningsdata kan vara orsak till underanpassning och att en lösning på problemet helt enkelt kan vara att öka mängden information att lära från.

Problemet med begränsade datamängder inom maskininlärning är ett vanligt problem. Naser (2026) framhåller att det är ett problem inom många områden och för många applikationer. En applikation kan vara för specialiserad för att det ska vara möjligt att samla in mer data eller det kan vara väldigt dyrt att samla in mer data. Kokol m. fl. (2022) tar specifikt upp att många forskare har problemet att de bara har tillgång till begränsade datamängder.

Att träna modeller på begränsade datamängder behöver dock inte bara vara ett problem. Det kräver för det första mindre resurser i form av energi och hårdvara (Naser, 2026). I mycket specialiserade applikationer kan modellerna dessutom fungera bättre än modeller som är tränade på större men mer generiska datamängder.

2.1.2 Hantering av problemet med överanpassning

Det finns flera sätt att reducera problemet med överanpassning. Ett sätt är att anpassa modellerna genom regularisering (Deisenroth m. fl., 2020, s. 235). Regularisering handlar om att man inför en form av sanktion mot att skapa för komplexa modeller som kan leda till just överanpassning. I fallet då modellen är probabilistisk, så korresponderar idén om regularisering till prior-sannolikhet (Deisenroth m. fl., 2020, s. 236–242). Men man kan också beräkna en så kallad posterior distribution där man tar hänsyn till observerad data och kan beräkna något som kallas maximum posterior uppskattning (maximum a posteriori estimation).

För att få en uppfattning om huruvida den framtagna modellen är överanpassad, speciellt vid begränsad datamängd, så kan metoden korsvalidering (cross-validation) (Deisenroth m. fl., 2020, s. 236) användas vid träning och testning. Se avsnitt 2.7.2.

2.2 Låg datakvalitet

Det är inte bara en begränsad mängd träningsdata som kan vara ett problem vid maskininlärning, utan träningsdata kan också ha låg kvalitet. Exempel på sådana utmaningar ges i detta avsnitt.

2.2.1 Exempel på låg datakvalitet

Ett exempel på en egenskap som vi menar kan betraktas som låg datakvalitet är högdimensionellitet. Om data är högdimensionell innebär det, se Appendix B, att varje indatapunkt eller vektor har många attribut. Detta kan vara bekymmersamt av flera olika anledningar, men en anledning är att det kan leda till överanpassning (Altman & Krzywinski, 2018). Anledningen till detta är att modellernas flexibilitet, vilket kan tolkas som hur flexibla de är gällande att anpassa sig till träningsdata, delvis beror av hur många variabler som ingår. Problemet med högdimensionell data har till och med ett eget namn, det benämns dimensionalitetens förbannelse (the curse of dimensionality).

Ett annat problem kan vara att träningsdata är obalanserad. Lindholm m. fl. (2022, s. 87–88) menar att ett problem är obalanserat om den stora övervägande majoriteten data tillhör den ena klassen. Problemet är mer specifikt att en modell tränad på sådan data kan förutse att ny data tillhör denna större klass och det kommer att se ut som om modellen presterar väl därför att den inte kommer att göra många felklassificeringar.

Ytterligare ett problem kan vara att träningsdata är brusig. Deisenroth m. fl. (2020, 6, egen översättning) menar visserligen att data kan ses just som ”brusiga observationer av någon sann underliggande signal” och att maskininlärning används just för att kunna urskilja signalen i bruset. Men om man har mycket brus, så är det möjligt att en flexibel modell också kommer att anpassas till bruset och det innebär överanpassning (Deisenroth m. fl., 2020, s. 243).

2.2.2 Hantering av låg datakvalitet

Gällande högdimensionell data, så kan det vara så att vissa dimensioner av data är överflödiga och det kan vara möjligt att istället bearbeta mer lågdimesionell data (Deisenroth m. fl., 2020, s. 286). Ett exempel på en metod för att åstadkomma detta är principalkomponentanalys (principal component analysis, PCA). När det kommer till obalanserad träningsdata, så är det möjligt att modifiera modellen, eller snarare den förlustfunktion som används för att ta hänsyn till att data är obalanserad (Lindholm m. fl., 2022, s. 101). Det är också möjligt att göra olika typer av urval av träningsdata, se avsnitt 2.7.1. När det kommer till att göra data mindre brusig, kan man preparera den data som används på olika sätt, se avsnitt 2.3.

2.3 Förbehandling av textdata

Inom textklassificering genomgår textdata oftast en förbehandlingsprocess för att reducera brus och därmed förbättra modellens prestanda (Siino m. fl., 2023). Vanliga steg inkluderar borttagning av specialtecken, konvertering av text till gemener (lowercasing) samt filtrering av fyllnadsord och andra icke-informativa termer. Tekniker som lemmatisering kan även användas för att reducera ord till deras grundform och därigenom minska variationen i data. Vidare kan man slå ihop ord med liknade betydelse, till exempel genom agglomerativ klustring som beskrivs nedan i avsnitt 2.3.1.

Samtidigt visar studier att nyttan av dessa metoder varierar beroende på modell och datamängd. Traditionella maskininlärningsmodeller tenderar att gynnas mer av omfattande förbehandlingssteg. Medan moderna transformerbaserade modeller bygger på kontextuell förståelse och relationer mellan ord, vilket gör att de i vissa fall påverkas mindre eller till och med negativt av sådana metoder (Siino m. fl., 2023).

2.3.1 Agglomerativ klustring

Agglomerativ klustring är ett sätt att gruppera information, vilket i denna studie handlar om att gruppera ord utifrån semantisk likhet (Lodhi m. fl., 2023). Varje ord representeras som en numerisk vektor i ett flerdimensionellt semantiskt rum, där ord med liknande betydelse ligger nära varandra. Inledningsvis behandlas varje ordvektor som ett enskilt kluster, därefter beräknar man avståndet mellan klustren genom euklidisk distans för att approximera likheten mellan ord, enligt formeln:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Man kan dessutom normalisera vektorerna som representerar orden till enhetsvektorer. Detta innebär att den euklidiska distansen mellan vektorerna blir monotont relaterad till cosinuslikhet (Bouhsine m. fl., 2026). Det betyder att ord med hög cosinuslikhet får liten euklidisk distans, och vice versa. Cosinuslikhet är ett mått som ofta används för att beskriva semantisk likhet mellan ord.

Efter att distansen mellan orden har blivit beräknad slår man ihop de två kluster av ord med minst distans mellan sig. Det vill säga som har stora likheter mellan varandra. Sedan beräknar man återigen distansen och hela processen upprepas tills man nått en förutbestämd mängd av kluster eller maximal distans. Slutresultatet är en mängd kluster där ord med liknande betydelse är grupperade. Detta kan sedan användas för att ersätta synonymer med en gemensam form. I och med detta reduceras synonymer och variationer av ett ord till en representation i data.

2.4 Textrepresentation

För att kunna bearbeta textinnehåll med maskininlärningsmetoder måste text först omvandlas till numeriska representationer. Detta görs genom att representera varje dokument som en vektor i ett numeriskt rum, där varje dimension motsvarar en term (t.ex. ett ord eller en symbol). I detta avsnitt beskrivs två klassiska metoder: Påse med ord (Bag of words, BoW) och Termfrekvenser-invers dokumentfrekvenser (Term Frequency-Inverse Document Frequency, TF-IDF), samt hur dessa kan utökas med n-gram.

2.4.1 Påse med ord (BoW)

Antag en vokabulär bestående av n unika termer $\{w_1, \dots, w_n\}$. I BoW representeras ett dokument som en frekvensvektor, där varje komponent anger antalet förekomster av en term i vokabulären (Jurafsky och Martin, 2026b). Dokument d representeras då av frekvensvektorn

$$\mathbf{f}^{(d)} = (f_1^{(d)}, f_2^{(d)}, \dots, f_n^{(d)}),$$

där $f_i^{(d)} \in \mathbb{N}_0$ anger antalet förekomster av termen w_i i dokumentet. Representationen bortser från termernas inbördes ordning. Denna representation tenderar att ge upphov till glesa och högdimensionella vektorer, eftersom varje unik term i vokabulären motsvarar en egen dimension.

2.4.2 Termfrekvenser-invers dokumentfrekvenser (TF-IDF)

TF-IDF-representationen bygger vidare på BoW genom att vikta termfrekvensen $f_i^{(d)}$ med den inversa dokumentfrekvensen. Detta innebär att termer som förekommer frekvent i många dokument tilldelas lägre vikt, medan mer särskiljande termer får högre vikt (Manning m. fl., 2008). För en term w_i definieras den inversa dokumentfrekvensen som

$$\text{idf}(w_i) = \log \left(\frac{N}{\text{df}_i} \right),$$

där N är det totala antalet dokument och df_i är antalet dokument där termen w_i förekommer. I praktiska implementationer används ofta en utjämnad variant (scikit-learn developers, 2026e): $\text{idf}(w_i) = \log \left(\frac{N+1}{\text{df}_i+1} \right) + 1$. Dokument d representeras då av vektorn $\mathbf{v}^{(d)} = (v_1^{(d)}, \dots, v_n^{(d)})$, där varje komponent definieras som

$$v_i^{(d)} = f_i^{(d)} \cdot \left(\log \left(\frac{N+1}{\text{df}_i+1} \right) + 1 \right).$$

I praktiken normaliseras ofta vektorn $\mathbf{v}^{(d)}$ med L2-normalisering (den euklidiska normen):

$$\mathbf{v}_{\text{norm}}^{(d)} = \frac{\mathbf{v}^{(d)}}{\|\mathbf{v}^{(d)}\|_2} = \frac{\mathbf{v}^{(d)}}{\sqrt{\sum_{i=1}^n (v_i^{(d)})^2}}.$$

Normaliseringen motverkar att längre dokument får oproportionerligt stora vektorlängder enbart på grund av att de innehåller fler termer, vilket möjliggör mer rättvisande jämförelser mellan dokument.

2.4.3 N-grams

Eftersom varken BoW eller TF-IDF tar någon hänsyn till ordningen på orden i en text, så kan n-grams användas för att sätta dem i ett sammanhang. Metoden fungerar så att alla sekvenser av n på varandra följande ord inkluderas som enskilda termer och är med i bedömningen av vilken klass dokumentet tillhör (scikit-learn developers, 2026a).

Alltså kan modellen skilja på om det står ”mycket bra” eller ”inte bra”, vilket kan vara en fördel för att upptäcka i vilken kontext ord används. Att använda n-grams ger en lokal kontext, men

upptäcker inte något långtgående beroende då ordningen på n-grams inte tas hänsyn till (scikit-learn developers, 2026a). N-grams kan användas i kombination med varandra, alltså att man kollar på till exempel unigrams (n=1) och bigrams (n=2) samtidigt.

2.5 Klassificeringsmodeller

För textklassificering används vanligtvis övervakade maskininlärningsmetoder (Jurafsky och Martin, 2026a). Vid övervakad inläring tränas en modell på en datamängd där varje dokument är annoterat med en klass. Modellen lär sig då sambandet mellan dokumentens textrepresentationer och deras klasstillhörighet, vilket gör det möjligt att klassificera nya dokument. I detta avsnitt beskrivs de tre klassificeringsmodellerna Naiv Bayes, logistisk regression och stödvektormaskin.

2.5.1 Naiv Bayes

Naiv Bayes är en probabilistisk klassificeringsmetod baserad på Bayes sats (Jurafsky och Martin, 2026b). Namnet *naiv* syftar på modellens oberoendeantagande mellan termerna givet dokumentets klass. Trots att antagandet sällan är helt uppfyllt fungerar modellen ofta väl i praktiken (Zhang och Gao, 2011). Klassificeringen sker genom att välja den klass c som har högst posterior sannolikhet givet dokumentrepresentationen $\mathbf{x}^{(d)}$ (Jurafsky och Martin, 2026b):

$$\hat{c} = \arg \max_c P(c | \mathbf{x}^{(d)}).$$

Den posteriora sannolikheten beräknas enligt Bayes sats:

$$P(c | \mathbf{x}^{(d)}) = \frac{P(\mathbf{x}^{(d)} | c) P(c)}{P(\mathbf{x}^{(d)})},$$

där $P(\mathbf{x}^{(d)} | c)$ är likelihooden, $P(c)$ är prior-sannolikheten för klass c och $P(\mathbf{x}^{(d)})$ är den totala sannolikheten att observera dokumentrepresentationen.

För att beräkna likelihooden krävs en sannolikhetsmodell för hur dokument genereras givet en klass. Vid textklassificering används ofta *Multinomial Naive Bayes*, som antar att termerna i ett dokument genereras villkorligt oberoende givet dokumentets klass enligt en multinomialfördelning. Modellen är formellt definierad för diskreta termräkningar. För ett dokument d betecknar $x_i^{(d)}$ antalet förekomster av termen w_i , och V beteckna antalet unika termer i vokabulären. Likelihooden kan då skrivas som (Manning m. fl., 2009):

$$P(\mathbf{x}^{(d)} | c) = \underbrace{\frac{\left(\sum_{i=1}^V x_i^{(d)}\right)!}{\prod_{i=1}^V x_i^{(d)}!}}_{\text{multinomialkoefficient}} \underbrace{\prod_{i=1}^V P(w_i | c)^{x_i^{(d)}}}_{\text{termsannolikheter}}. \quad (1)$$

Termsannolikheten $P(w_i | c)$ uppskattas från träningsdata enligt $P(w_i | c) = \frac{N_{ic}}{N_c}$, där (scikit-learn developers, 2026d):

$$N_{ic} = \sum_{d \in c} x_i^{(d)}, \quad N_c = \sum_{j=1}^V \sum_{d \in c} x_j^{(d)}. \quad (2)$$

För en BoW-representation motsvarar N_{ic} det totala antalet förekomster av termen w_i i alla träningsdokument som tillhör klass c , medan N_c anger det totala antalet termförekomster i samtliga träningsdokument i klass c . I praktiska tillämpningar används modellen dock ofta tillsammans med TF-IDF-representationer (Dewi m. fl., 2023). I detta fall motsvarar $x_i^{(d)}$ termens TF-IDF-vikt, och N_{ic} samt N_c beräknas som summor av dessa vikter. TF-IDF uppfyller därmed inte strikt den multinomiala modellens antagande om diskreta termräkningar. Trots detta har kombinationen av Multinomial Naive Bayes och TF-IDF visat god klassificeringsprestanda i praktiken (Blog, 2022).

Om en term saknas i träningsdokumenten för en viss klass gäller $N_{ic} = 0$, vilket ger $P(w_i | c) = 0$ och därmed en likelihood på noll. För att undvika detta används additiv utjämning (Laplace-utjämning), där en positiv parameter α adderas till varje termfrekvens (scikit-learn developers, 2026c). Termsannolikheterna beräknas då som:

$$P(w_i | c) = \frac{N_{ic} + \alpha}{N_c + \alpha V}. \quad (3)$$

Parametern $\alpha > 0$ styr graden av utjämning. Större värden ger starkare utjämning och mindre skillnader mellan termsannolikheterna, medan ett mindre värde gör modellen mer beroende av de observerade mönstren i träningsdata.

Eftersom både $P(\mathbf{x}^{(d)})$ och multinomialkoefficienten är oberoende av klassen c , kan de utelämnas vid klassificeringen: $\hat{c} = \arg \max_c P(c) \prod_{i=1}^V P(w_i | c)^{x_i^{(d)}}$. I praktiska implementationer används logaritmer för att undvika numeriska problem som uppstår vid multiplikation av många små sannolikheter. Produkten omvandlas då till en summa:

$$\hat{c} = \arg \max_c \left[\underbrace{\log P(c)}_{\text{log-prior}} + \sum_{i=1}^V \underbrace{x_i^{(d)} \log P(w_i | c)}_{\text{log-likelihood}} \right]. \quad (4)$$

Varje term bidrar därmed additivt till en total log-sannolikhet. För BoW-representationer bestäms bidraget av termens frekvens i dokumentet och dess sannolikhet inom respektive klass. Vid användning av TF-IDF motsvarar $x_i^{(d)}$ termens TF-IDF-vikt, vilket innebär att mer särskiljande termer får större inflytande på den totala log-sannolikheten. Dokumentet klassificeras slutligen till den klass som erhåller högst total log-sannolikhet.

2.5.2 Logistisk regression

Logistisk regression är en linjär probabilistisk modell som ofta används för binär klassificering. Den grundläggande formuleringen är avsedd för två klasser, men modellen kan även utökas till flerklassproblem (Jurafsky och Martin, 2026a). För att utföra klassificering beräknar modellen först en linjärkombination baserad på dokumentets representation $\mathbf{x}^{(d)}$:

$$z^{(d)} = \alpha + \boldsymbol{\beta}^\top \mathbf{x}^{(d)} = \alpha + \sum_{i=1}^n \beta_i x_i^{(d)}.$$

där parametrarna α och β_i lärs från träningsdata. Vikten β_i anger termens bidrag till klassificeringsbeslutet. Positiva värden ökar sannolikheten för den ena klassen, medan negativa värden ökar sannolikheten för den andra. Ju större absolutvärde $|\beta_i|$ har, desto större påverkan har termen. Parametern α är en konstant (intercept) som förskjuter modellens beslutgräns och gör att modellen kan anpassa sannolikheten även när alla indata är noll (Stanford University, n.d.).

Sannolikheten beräknas därefter med sigmoidfunktionen, även kallad den logistiska funktionen:

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

vilket har gett upphov till namnet logistisk regression (Jurafsky och Martin, 2026a). Funktionen omvandlar den linjära kombinationen till en sannolikhet i intervallet $[0, 1]$. Vid binär klassificering representerar den binära variabeln $y \in \{0, 1\}$ dokumentets klass, där exempelvis $y = 1$ betecknar den positiva klassen och $y = 0$ den negativa klassen. Eftersom sannolikheterna för de två klasserna summerar till 1 räcker det att modellera sannolikheten för den ena klassen. Den predikterade sannolikheten att dokument d tillhör klassen $y = 1$ ges därmed av

$$p^{(d)} = P(y = 1 | \mathbf{x}^{(d)}) = \sigma(\boldsymbol{\beta} \cdot \mathbf{x}^{(d)} + \alpha) = \frac{1}{1 + e^{-(\boldsymbol{\beta} \cdot \mathbf{x}^{(d)} + \alpha)}}. \quad (5)$$

Eftersom sigmoidfunktionen antar värden i intervallet $[0, 1]$ kan sannolikheten användas för klassificering genom att införa ett tröskelvärde, exempelvis 0,5 som motsvarar ett antagande om lika priorer mellan klasserna. Den predikterade klassen ges då av

$$\hat{y} = \begin{cases} 1 & \text{om } p^{(d)} \geq 0,5; \\ 0 & \text{annars.} \end{cases} \quad (6)$$

Parametrarna β och α estimeras genom att minimera den så kallade korsentropiförlusten, vilken fungerar som modellens förlustfunktion:

$$J(\beta, \alpha) = - \sum_{d=1}^N \left[y^{(d)} \log p^{(d)} + (1 - y^{(d)}) \log(1 - p^{(d)}) \right]. \quad (7)$$

Här representerar $y^{(d)} \in \{0, 1\}$ den sanna klassen för dokument d , medan $p^{(d)}$ är modellens predikterade sannolikhet att dokumentet tillhör klassen $y = 1$. Förlustfunktionen mäter skillnaden mellan modellens prediktioner och de faktiska klassetiketterna. Höga förluster uppstår när modellen tilldelar låg sannolikhet till den korrekta klassen, medan låga förluster erhålls när modellen gör korrekta och säkra prediktioner. Genom att minimera korsentropiförlusten lär sig modellen parametrar som ger så korrekta sannolikhetsskattningar som möjligt.

För att motverka överanpassning används ofta regularisering. Exempelvis L2-regularisering:

$$J_{\text{reg}} = J + \lambda \|\beta\|^2, \quad (8)$$

där λ är styrkan på regulariseringen. I många praktiska implementationer, såsom i Scikit-learn, introduceras istället parametern $C = \frac{1}{\lambda}$ (scikit-learn developers, 2026b). Detta ger $J_{\text{reg}} = J + \frac{1}{C} \|\beta\|^2$. Ett litet värde på C innebär stark regularisering, vilket tvingar vikterna β att vara små. Detta leder till en enklare modell med risk för underanpassning, eftersom modellen inte utnyttjar informativa termer fullt ut. I praktiken innebär detta att skillnaden mellan vikterna för informativa och icke-informativa termer minskar. Ett stort värde på C innebär däremot svag regularisering, vilket tillåter större vikter. Då bestäms vikterna i högre grad av träningsdatan, vilket ger en mer flexibel modell med bättre anpassning till träningsdatan, men samtidigt en ökad risk för överanpassning.

2.5.3 Stödvektormaskin (SVM)

Modellen stödvektormaskin (support vector machine, SVM) handlar om att optimera en funktion och innebär ett geometriskt synsätt (Deisenroth m.fl., 2020, s. 336). Den exakta utformningen kan dock se ut på olika sätt och modellen kan i själva verket betraktas som en familj av modeller (Bonaccorso, 2018, s. 207). Den data som modellen ska tränas på påverkar exakt utformning. Om data är möjlig att separera linjärt kan modellen se ut på ett sätt, annars kan justeringar behöva göras.

SVM för linjärt separerbar data. Den grundläggande idén är att den data som ska klassificeras representeras i form av vektorer och målet är att hitta det hyperplan som bäst separerar vektorerna i de olika klasserna åt (Bonaccorso, 2018, s. 207–208). Grundekvationen för detta hyperplan, där $\mathbf{x}$ är de vektorer som ska klassificeras, är:

$$\mathbf{w}^T \cdot \mathbf{x} + b = 0$$

där $\mathbf{w} = (w_1, w_2, \dots, w_m)^T$. Detta är ett plan som är linjärt och kommer alltså endast att på ett fullständigt korrekt sätt kunna separera data som är möjlig att separera med just ett sådant plan. Man kan också matematiskt se det som att man delar (partition) rummet där data finns representerad med hjälp av hyperplanet, vilket är ett underrum med en dimension lägre än hela rummet (Deisenroth m.fl., 2020, s. 336–337).

Det hyperplan som bäst separerar vektorerna är enkelt uttryckt det hyperplan som har störst

marginal till datapunkterna. Mer specifikt tänker man sig två andra hyperplan som representerar gränserna för dessa marginaler, där målet är att maximera avståndet mellan dessa plan (Bonaccorso, 2018, s. 209). För på så sätt minskar man sannolikheten för att något klassificeras fel. De vektorer som ligger på dessa två hyperplan kallas stödvektorer och det är deras roll som har givit modellen dess namn. Hyperplanen har följande ekvationer:

$$\begin{cases} \mathbf{w}^T \cdot \mathbf{x} + b = -1 \\ \mathbf{w}^T \cdot \mathbf{x} + b = 1 \end{cases}$$

Genom beräkningar av avståndet mellan de hyperplan som representerar gränserna för marginalen (hädanefter kallade marginalplanen) och genom villkor för hur stort detta avstånd minst måste vara, så nås följande funktion att minimera för att maximera avståndet mellan marginalplanen (Bonaccorso, 2018, s. 210–211):

$$\min \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

där $y_i(\mathbf{w}^T \cdot \mathbf{x}_i + b) \geq 1 \forall (\mathbf{x}_i, y_i)$ och där $y_i \in \{-1, 1\}$ representerar klassetiketten för datapunkt i (Bonaccorso, 2018, s. 208).

SVM för nästan linjärt separerbar data (C-SVM). Men all data är inte möjlig att separera linjärt. En del data kan dock vara nästan möjlig att separera linjärt och dessa situationer kan hanteras genom så kallad C-SVM. Grundtanken med C-SVM är att skapa ett mer flexibelt hyperplan som tillåter att vissa vektorer som tillhör en viss klass kan befinna sig på fel sida om hyperplanet (Bonaccorso, 2018, s. 212). Hur flexibelt hyperplanet ska vara bestäms genom en så kallad C-parameter, där större värden leder till mer flexibilitet men också till ökad felklassificering. Det C-parametern mer specifikt gör är att den åstadkommer en avvägning mellan hur stor marginalen är och mängden "slack" och den kallas regulariseringsparameter (Deisenroth m. fl., 2020, s. 344). Mängden slack anges av den så kallade slackvariabeln ξ_i , vilken alltså tillåter en datapunkt att befinna sig inom marginalen eller på fel sida av hyperplanet (Deisenroth m. fl., 2020, s. 344). C-SVM kallas också mjuk marginal-SVM (soft margin SVM) (Deisenroth m. fl., 2020, s. 343). Funktionen som i fallet C-SVM ska minimeras framkommer nedan (Bonaccorso, 2018, s. 212):

$$\min \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_i \xi_i$$

där $y_i(\mathbf{w}^T \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \forall (\mathbf{x}_i, y_i)$ och $\xi_i \geq 0$.

Gällande generaliserbarhet, så ger låga värden på C en stor marginal. Det leder till att hyperplanet på ett bättre sätt hittar mönstret i data, samtidigt som mer träningsdata kan felklassificeras. Detta kan leda till underanpassning. När värdet på C stiger, så blir marginalen mindre och det gör att mer träningsdata klassificeras rätt. Detta kan leda till överanpassning.

Det går också att tänka i termer av en förlustfunktion, då förlusten antar värdet 0 om en modells förutsägelse ligger på rätt sida om hyperplanet och utanför marginalen, men positiva värden om innanför marginalen eller på fel sida om hyperplanet (Deisenroth m. fl., 2020, s. 345). Tanken är sedan att minimera den totala förlusten och då kan minimeringsproblemet uttryckas som i ekvation 9 (Deisenroth m. fl., 2020, s. 346). I implementationen av linjär SVM i scikit-learn (LinearSVC), se avsnitt 3.4, används dock kvadratisk förlust (förlusttermen upphöjd till 2) som standard.

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \max\{0, 1 - y_n(\langle \mathbf{w}, \mathbf{x}_n \rangle + b)\} \quad (9)$$

SVM för icke linjärt separerbar data. I de fall där data inte alls liknar linjärt separerbar data, kan istället mer avancerade så kallade kärnor (kernels) användas. Tanken är att vektorerna projiceras till ett annat rum, ofta med högre dimension, där de just är möjliga att separera linjärt (Bonaccorso, 2018, s. 215). Fallet med icke alls linjärt separerbar data kommer dock inte att behandlas i denna studie och kärnor beskrivs därför inte ytterligare.

2.6 Prestandamått

Några etablerade prestandamått som används för utvärdering av klassificeringsmodeller är noggrannhet (accuracy), precision, känslighet/klassvis noggrannhet (recall) och F1-poäng (F1-score) (Jurafsky & Martin, 2026a).

Tabell 3: Prestandamått och deras formler.

Prestandamått	Formel
Noggrannhet	$\text{Noggrannhet} = \frac{\sum_{i=1}^n TP_i}{m}$
Precision	$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$
Makroprecision	$\text{Precision}_{\text{makro}} = \frac{\sum_{i=1}^n \text{Precision}_i}{n}$
Känslighet/Klassvis noggrannhet	$\text{Känslighet}_i = \frac{TP_i}{N_i}$
Makrokänslighet	$\text{Känslighet}_{\text{makro}} = \frac{\sum_{i=1}^n \text{Känslighet}_i}{n}$
F1-poäng	$F1_i = \frac{2 \cdot \text{Känslighet}_i \cdot \text{Precision}_i}{\text{Känslighet}_i + \text{Precision}_i}$
Makro F1-poäng	$F1_{\text{makro}} = \frac{\sum_{i=1}^n F1_i}{n}$

I tabell 3 ovan visas formlerna för de olika prestandamåtten. Ett makrogenomsnitt innebär att varje klass bidrar med samma vikt till resultatet, oberoende av klassens storlek (Grandini, M., Bagli, E., and Visani, G., 2020). I formlerna representerar TP_i de fall där modellen har klassificerat rätt för klass i , n är antal klasser, m är det totala antal dokument i datamängden och N_i är antal dokument tillhörande klass i . FP_i representerar de fall där klassificeringen felaktigt tilldelat ett dokument klass i . För en mer utförlig presentation av prestandamåtten, se Appendix D.

2.7 Träning och testning

För att träna och testa klassificeringsmodeller används olika metoder. I detta avsnitt beskrivs olika samplingstekniker och hur k-faldig korsvalidering kan användas.

2.7.1 Sampling

Vid klassificeringsproblem förekommer ofta obalanserade datamängder, där vissa klasser innehåller betydligt fler datapunkter än andra (Padurariu & Breaban, 2019). Detta kan leda till att modellen under träning anpassar sig främst efter majoritetsklassen och får svårare att sedan korrekt identifiera minoritetsklassen. Ett vanligt sätt att hantera detta problem är att justera klassfördelningen i träningsdata genom olika omsamplingstekniker, såsom översampling eller undersampling (Baladram, 2024). Översampling innebär att utöka minoritetsklassen genom att generera ny data eller skapa kopior av existerande data, ofta slumpvis för att minska risken för överanpassning. Undersampling innebär istället att man tar bort data från majoritetsklassen för att skapa en mer balanserad fördelning.

2.7.2 K-faldig korsvalidering

K-faldig korsvalidering (k-fold cross-validation) är en metod för att effektivt utnyttja små datamängder (Jurafsky och Martin, 2026a). Metoden delar upp datamängden i k delmängder (folds), där $k - 1$ delmängder används för träning medan den återstående delmängden används som testmängd. Proceduren upprepas tills varje delmängd har använts exakt en gång som testmängd. Detta tillvägagångssätt innebär att tränings- och testdata hålls separerade i varje iteration, samtidigt som hela datamängden har använts för både träning och testning efter samtliga iterationer. Metoden är särskilt användbar vid begränsade datamängder eftersom den möjliggör ett effektivt utnyttjande av tillgänglig data och ger k olika uppskattningar av modellens prestanda. Dessa uppskattningar

kan sedan kombineras för att erhålla en mer robust skattning av modellens prestation och generaliseringsförmåga. En variant av metoden är stratifierad k-faldig korsvalidering, där varje delmängd bevarar klassfördelningen från den ursprungliga datamängden (Mahesh m. fl., 2023).

3 Implementation

I detta kapitel redogörs för det praktiska utförandet av studien. Implementationen innefattade förbehandling av data, urval av tränings- och testmängder samt modellträning med olika textrepresentationer. Olika prestandamått för utvärdering av de framtagna klassificeringsmodellerna har också utvärderats. Implementeringen utfördes i programmeringsspråket Python (version 3.12.0). Källkoden för implementeringen återfinns i Appendix J.

3.1 Beskrivning av data

Den data som studien hade tillgång till för träning och testning av klassificeringsmodellerna var 448 intervjuer om anledning till flytt från bostäder, producerade inom ramen för Chalmers verksamhet. Intervjuerna hade formen av Microsoft Word-dokument namngivna med unika identifierare (ID). Dessa dokument var indelade i två övergripande klasser, relaterade till om renovering var flyttorsaken eller inte. De två klasserna var obalanserade, vilket framkommer i tabell 4 nedan, som visar hur många dokument som tillhörde respektive klass.

Tabell 4: Antal dokument tillhörande respektive klass.

Klass	Antal dokument
Ingen koppling till renovering	331
Koppling till renovering	117

Kategorin *Ingen koppling till renovering* är egentligen en samling av sex mindre kategorier med andra flyttorsaker än renovering. Men i denna studie har de betraktats som en enda kategori. Eftersom de intervjuade angav flera olika anledningar till flytt i majoritetsklassen, och eftersom intervjuerna bedömdes innehålla mycket information som inte är direkt relaterad till frågan om vilken klass de ska tillhöra, betraktade vi data som brusig. Dessutom innehöll data totalt ca 10.000 unika termer, vilket vi betraktade som högdimensionellt. Med anledning av detta genomfördes en del arbete med att förbehandla data där målet var att göra den mindre brusig, men också mindre högdimensionell. För mer detaljerade frågor om den datamängd som använts, kontakta projektets handledare.

3.2 Förbehandling av data

Vid genomgång av data uppdagades att dokumentens uppbyggnad varierade. Ungefär hälften av dokumenten utgjorde sammanfattningar av intervjuer medan den andra hälften av dokumenten utgjorde transkriptioner med intervjufrågor och svar. Därav blev första steget att extrahera intervjuvaren från transkriptionerna och sammanfattningarna sparades i sin helhet. Därefter rensades dokumenten på specialtecken och metadata. Sedan konverterades orden till gemener. Dessa steg utfördes med hjälp av Python-biblioteket `docx` och resultatet sparades i en json-fil vid namn *raw_textdata*, vilken fungerade som utgångspunkt vid träningen av klassificeringsmodellerna.

Trots att mycket text nu rensats bort var data fortfarande mycket högdimensionell och brusig. Det ledde till nästa steg i förbehandlingen av data, vilket var att filtrera bort stoppord, det vill säga ord med liten signifikans för klassificeringen. För att göra detta användes Python-biblioteket *NLTK* och dess lista med svenska stoppord. Resultatet sparades i en ny json-fil vid namn *stopordsfiltrerad*. Nästa insats för att förbättra datakvaliteten var att lemmatisera orden genom Python-biblioteket *spaCy*, och detta sparades i en ny json-fil med namnet *lemmatiserad*.

Avslutningsvis användes Python-biblioteken *sentence_transformers* och *scikit-learn* för att utföra agglomerativ klustring och skapa en mappning (map) av synonymer, som sedan användes för att

transformera flera ord till en gemensam term. Detta hade även en ytterligare effekt, då ett flertal felstavningar som förekom i dokumenten korrigerades till sin korrekta stavning. Resulterande data sparades i en slutgiltig json-fil kallad *synonymnormaliserad*. Alla json-filer bygger på varandra så denna slutliga fil innehöll även alla tidigare åtgärder som gjorts.

3.3 Urval av tränings- och testmängder

Studien undersökte två urvalsstrategier för tränings- och testdata:

1. Proportionell fördelning: Stratifierad femfaldig korsvalidering användes, där datamängden delades upp i fem delmängder med bibehållen klassfördelning. Vid varje iteration användes fyra delmängder för träning och en delmängd för testning.
2. Balanserad fördelning: Samma stratifierade femfaldiga korsvalidering användes, men träningsmängden balanserades sedan genom slumpmässig sampling av exakt 93 dokument från varje klass (vilket motsvarar 80 % av minoritetsklassen). Testmängden behöll samtidigt sin ursprungliga klassfördelning.

Vid denna undersökning av urval användes modellernas standardvärden för hyperparametrarna, vilka anges i avsnitt 3.4 nedan.

3.4 Modellträning och testning med olika textrepresentationer

Vid modellträning och testning användes klassificeringsmodeller samt textrepresentationer importerade från Python-biblioteket *scikit-learn*. För att lätt kunna jämföra modellerna med varandra och för att hitta bästa möjliga prestanda testades alla modeller i kombination med de båda utvalda textrepresentationerna:

- BoW: här användes `CountVectorizer(max_df=1,0; min_df=1, ngram_range = (1,1))`
- TF-IDF: här användes `TfidfVectorizer (max_df=1,0; min_df=1, ngram_range = (1,1))`

Modellerna som användes var:

- För multinomial Naiv Bayes: `MultinomialNB( $\alpha = 1, 0$ )`
- För SVM: `LinearSVC1( $C = 0, 5$ ),`
- För logistisk regression: `LogisticRegression( $C = 1, 0$ )`

Inom parentes anges standardvärden för textrepresentationernas och modellernas parametrar.

Modellträningen utfördes genom tvåstegsoptimering. I första steget utvärderades kombinationer av *min_df*, *max_df* samt *n_grams* i textrepresentationerna tillsammans med modellernas standardvärden. Detta gjordes på alla json-filer med balanserad träningsmängd. *min_df*, som bestämmer hur många dokument en term minst måste förekomma i för att inkluderas i modellträningen, testades på intervallet [1;5]. *max_df* anger den högsta dokumentfrekvens en term får ha för att inkluderas i modellträningen och testades på [0,8; 0,9; 1,0]. *ngram_range* bestämmer vilka *n*-grams som används vid vektoriseringen, här testades (1,1) med endast unigrams och (1,2) med både unigrams och bigrams.

I andra steget testades istället olika värden på α och C i modellerna. Här användes de värden som i första steget maximerat prestandan för respektive json-fil, och träningsmängden var balanserad. För Naiv Bayes och logistisk regression testades 50 olika värden på α respektive C vilka låg mellan 10^{-3} och 10^3 . För SVM testades 30 värden på C mellan 10^{-3} och 10^3 . När ett bästa värde utsetts så testades modellen också på värden i nära anslutning till detta för att utse ett mer specifikt värde som presterade bäst för modellen. En klassvis noggrannhet på minst 50 procent behövde uppfyllas för att värdet på parametern skulle vara med i bedömningen. Med klassvis noggrannhet, även kallat känslighet, avses i denna rapport modellens förmåga att korrekt klassificera en specifik klass.

¹Det har antagits att data är approximativt linjärt separerbar, eftersom detta representerar ett enkelt fall.

3.5 Prestandamått

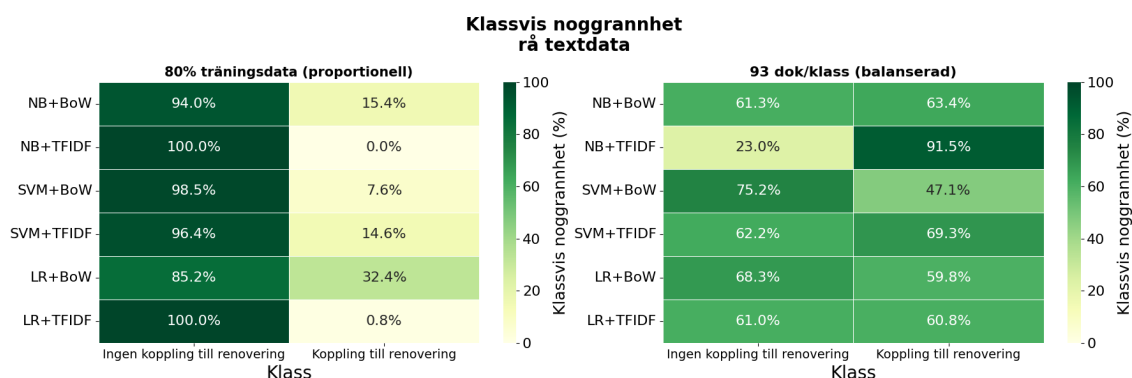
För att utvärdera hur väl modellerna presterade och vilken modell som presterade bäst användes prestandamåtteten noggrannhet, precision, känslighet/ klassvis noggrannhet och F1-poäng samt deras makroversioner. De mått som det lades störst vikt vid var makro F1-poäng och klassvis noggrannhet.

4 Resultat och diskussion

I detta kapitel presenteras studiens resultat och analysen av dessa. För enkelhetens skull betecknas Naiv Bayes med NB, logistisk regression med LR och stödvektormaskin med SVM. Först behandlas urvalsstrategierna för träningsdata, följt av de tre klassificeringsmodellerna och därefter en övergripande analys. Mer detaljerade resultat återfinns i appendixen, där bland annat analyser av prestandamått, fullständiga resultat för de bästa hyperparameterkombinationerna och ord med stor påverkan på modellernas klassificering redovisas.

4.1 Urvalsstrategier för träningsdata

Effekten av de två olika urvalsstrategierna för träningsdata analyserades genom att jämföra klassvis noggrannhet mellan proportionell och balanserad fördelning av träningsdata, med standardvärden på hyperparametrar och utan minimikrav på klassvis noggrannhet, se avsnitten 3.3- 3.4.



Figur 1: Klassvis noggrannhet för sex modell–textrepresentationskombinationer (modellkombinationer) på rå textdata. Vänster: proportionell fördelning. Höger: balanserad fördelning.

Figur 1 visar en tydlig skillnad i klassvis noggrannhet mellan de två urvalsstrategierna. Vid proportionell träningsdata uppnår de flesta metoder hög noggrannhet för majoritetsklassen *Ingen koppling till renovering* (cirka 84–100%), medan noggrannheten för minoritetsklassen *Koppling till renovering* i de flesta fall understiger 15%. Vid balanserad träning blir prestandan betydligt jämnare, där båda klasserna för majoriteten av modellkombinationerna uppnår en noggrannhet omkring 60–70%.

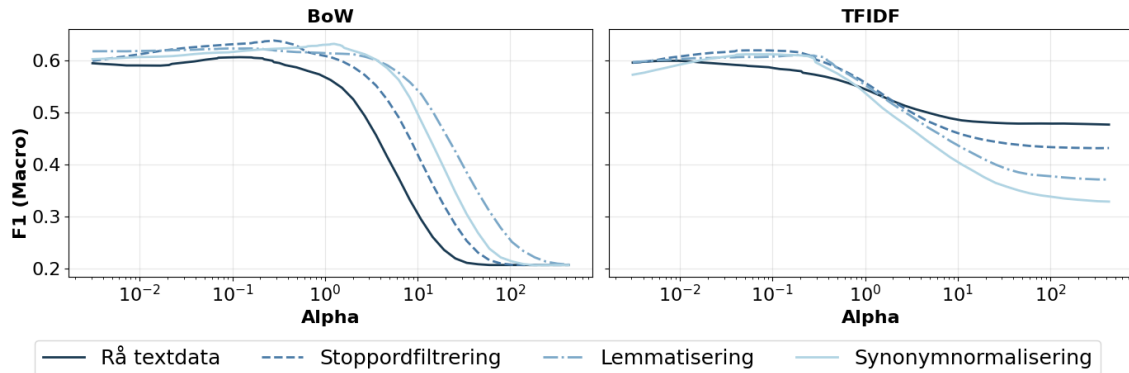
Resultaten visar att klassfördelningen i träningsdatan har stor påverkan på modellernas beteende. Vid träning på obalanserad data dominerar majoritetsklassen, vilket leder till hög noggrannhet för denna klass men betydligt sämre prestanda för minoritetsklassen. Detta kan förklaras av att modeller som logistisk regression och SVM optimerar en global förlustfunktion som minimerar det totala felet, vilket innebär att majoritetsklassen får större inflytande. För Naiv Bayes påverkas klassificeringen dels av klasspriorerna, dels av skattningen av termsannolikheterna. När en klass innehåller fler träningsdokument blir prior-sannolikheten för denna klass högre, vilket ökar sannolikheten att nya dokument klassificeras till majoritetsklassen. Samtidigt skattas termsannolikheterna utifrån fler observationer, vilket kan förstärka modellens benägenhet att favorisera majoritetsklassen.

När träningsdata istället balanseras reduceras denna snedvridning. Modellerna tvingas då i högre

grad lära sig att särskilja båda klasserna, vilket ger en jämnare klassvis noggrannhet. Minoritetsklassens prestanda förbättras markant, medan majoritetsklassens noggrannhet minskar något. Sammantaget visar resultaten vikten av att hantera klassobalans vid träning av klassificeringsmodeller, särskilt när målet är en jämn prestanda mellan klasser.

4.2 Naiv Bayes

I detta avsnitt analyseras hur utjämningsparametern α påverkar prestandan för Naiv Bayes i kombination med BoW respektive TF-IDF under olika förbehandlingssteg.



Figur 2: Makro F1-poäng som funktion av α för Naiv Bayes (vänster BoW, höger TF-IDF).

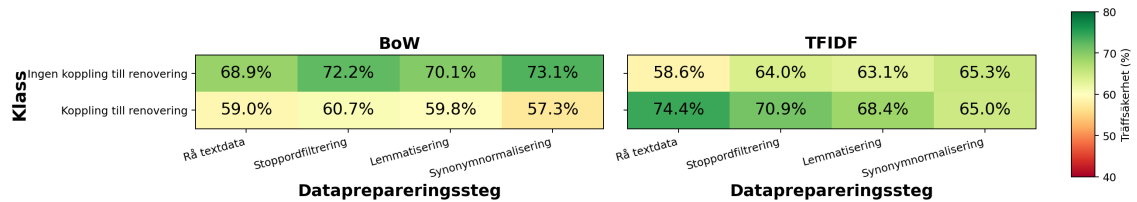
Figur 2 visar hur makro F1-poängen varierar med α . För BoW är prestandan relativt stabil i intervallet $0,001 \leq \alpha \leq 1$, där samtliga förbehandlingssteg uppnår sin maximala makro F1-poäng mellan 0,60–0,64. Vid större värden på α försämras prestandan markant: vid $\alpha = 100$ sjunker makro F1-poängen till cirka 0,2. Denna trend återfinns för samtliga förbehandlingssteg, men den kraftiga försämringen inträffar vid olika värden på α inom intervallet 1 till 10. Rå textdata påverkas först, följt av stoppordsfiltrering, synonymnormalisering och slutligen lemmatisering. För TF-IDF är kurvan generellt jämnare, precis som för BoW ligger den maximala makro F1-poängen för alla förbehandlingssteg på en liknande nivå, cirka 0,60–0,63. Prestandan börjar dock minska redan vid $\alpha \approx 0,5$. Vid större värden på α observeras en tydligare skillnad mellan förbehandlingsstegen, där prestandan minskar i takt med ökad grad av förbehandling. Vid $\alpha = 100$ uppnår rå textdata en makro F1-poäng på cirka 0,5; medan stoppordsfiltrering ger cirka 0,44; lemmatisering cirka 0,38 och synonymnormalisering cirka 0,34.

Den begränsade variationen i maximal makro F1-poäng indikerar att valet av förbehandlingssteg och textrepresentation har liten påverkan på den högsta uppnåbara prestandan, men större påverkan på modellens känslighet för utjämning. Den observerade försämringen i båda graferna vid stora värden på α kan förklaras av den kraftiga utjämning som införs i sannolikhetsskattningen i Naiv Bayes. För stora $\alpha V \gg N_c$ gäller approximativt att $P(w_i | c) \approx \frac{\alpha}{\alpha V} = \frac{1}{V}$, se ekvation (3). Detta gör att termsannolikheterna närmar sig en uniform fördelning, vilket minskar skillnaderna mellan klasserna och försämrar klassificeringen. Skillnaden mellan BoW och TF-IDF uppstår främst genom dokumentrepresentationen $x_i^{(d)}$ i log-likelihoodtermen i klassificeringsregeln i ekvation (4), eftersom klasspriorerna är identiska vid träning på balanserad data. För BoW motsvarar $x_i^{(d)}$ rena termfrekvenser, vilket gör att högfrekventa termer som förekommer i båda klasserna kan dominera log-likelihooden trots låg särskiljande förmåga. Tillsammans med utjämning leder det till en kraftigare prestandaförsämring. För TF-IDF nedvikts frekventa och mindre informativa termer, medan mer klassspecifika termer får större relativ betydelse. Detta gör modellen mindre känslig för stark utjämning, vilket kan förklara den mer gradvisa prestandaförsämringen jämfört med BoW.

Variationer i makro F1-poäng mellan förbehandlingsstegen i BoW vid stora värden på α kan förklaras av hur förbehandlingen påverkar mängden brus och förekomsten av generella högfrekventa termer i datamängden. Rå textdata innehåller många vanliga termer och variationer, exempelvis

stoppord och olika böjningsformer, vilket gör den mer känslig för stark utjämning. Förbehandlingssteg som stoppordsfiltrering, lemmatisering och synonymnormalisering reducerar däremot mängden brus och koncentrerar termfrekvenserna till färre och mer informativa termer, vilket gör dem mer robusta mot variation i α och förklarar varför prestandaförsämringen inträffar vid högre värden på α . Att lemmatisering uppvisar en senare prestandaförsämring än synonymnormalisering trots att synonymnormalisering i den tillämpade förbehandlingskedjan utförs efter lemmatisering, kan förklaras av hur metoderna påverkar informationsinnehållet. Lemmatisering reducerar böjningsvariationer samtidigt som semantiska skillnader i stor utsträckning bevaras. Synonymnormalisering slår däremot samman termer med liknande betydelse, vilket kan minska viktiga klassspecifika skillnader och göra representationen mindre informativ.

Till skillnad från BoW sjunker makro F1-poängen för TF-IDF med ökad grad av förbehandling vid kraftig utjämning. Eftersom TF-IDF bygger på att identifiera särskiljande termer kan ytterligare förbehandling reducera variationen och antalet unika termer i datamängden, samtidigt som utjämning driver termsannolikheterna mot en mer uniform fördelning, vilket kan försvaga de klassspecifika mönstren. Att rå textdata uppvisar högre makro F1-poäng vid stora α -värden tyder alltså på att den större variationen i ursprungsdata bidrar med fler särskiljande termer och därmed gör modellen mer robust mot stark utjämning.

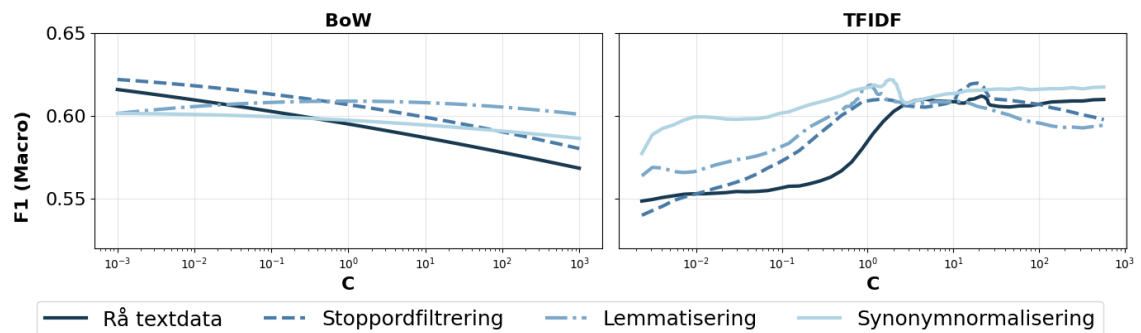


Figur 3: Klassvis noggrannhet för Naïv Bayes för olika förbehandlingssteg (vänster BoW, höger TF-IDF).

Figur 3 visar hur dessa effekter påverkar modellens klassificeringsbeteende. För BoW observeras en tydligt ojämn klassvis noggrannhet, där modellen konsekvent gynnar majoritetsklassen. För TF-IDF är den klassvisa noggrannheten betydligt mer jämn, och skillnaden mellan klasserna minskar ytterligare med ökad grad av förbehandling. Vid synonymnormalisering uppnås en nästan jämn klassificering mellan klasserna. Resultatet kompletterar tidigare analys att TF-IDF i större utsträckning fångar klassspecifika mönster än BoW, vilket leder till en mer balanserad klassificering.

4.3 Logistisk regression

I detta avsnitt analyseras hur regulariseringsparametern C påverkar prestandan för logistisk regression i kombination med BoW respektive TF-IDF under olika förbehandlingssteg.



Figur 4: Makro F1-poäng som funktion av C för logistisk regression (vänster BoW, höger TF-IDF).

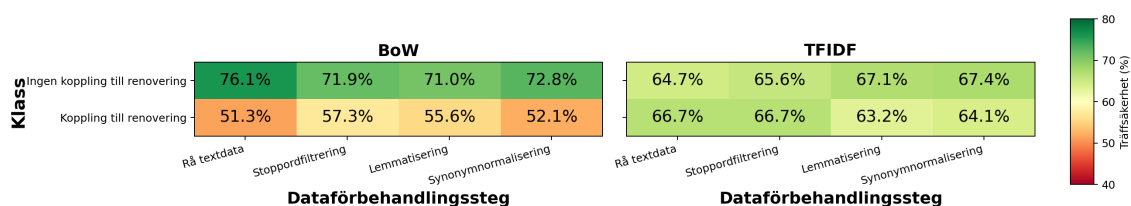
Figur 4 visar hur makro F1-poängen varierar med C . För BoW minskar prestandan för rå textda-

ta och stoppordsfiltrering när C ökar, medan lemmatisering samt synonymnormalisering uppvisar mindre variationer. För TF-IDF observeras ett motsatt mönster: makro F1-poängen ökar när C ökar från cirka 0,001 till 1. För rå textdata sker denna ökning senare med en brantare kurva kring $C \approx 1$, medan mer omfattande förbehandling leder till en tidigare ökning och högre initial prestanda. För samtliga förbehandlingssteg planar kurvorna ut vid $C \gtrsim 1$. För både BoW och TF-IDF ligger den maximala makro F1-poängen för samtliga förbehandlingssteg på liknande nivåer (cirka 0,60–0,62).

I likhet med resultaten för Naiv Bayes påverkar representationernas egenskaper modellernas känslighet för hyperparametrar, även om mekanismen här utgörs av regularisering snarare än utjämning. I enlighet med teorikapitlet (se avsnitt 2.5.2), där $C = \frac{1}{\lambda}$ kan dessa mönster förstås i termer av modellens komplexitet. Låga värden på C innebär stark regularisering och därmed en mer restriktiv modell, medan högre värden ger ökad flexibilitet.

Skillnaden mellan BoW och TF-IDF kan förklaras av samspelet mellan modellens regularisering och representationernas egenskaper. TF-IDF är relativt välbalanserad och normaliserad, vilket gör att modellen behöver mindre regularisering. Vid låga värden på C (t.ex. $C < 0,1$) pressas vikterna β mot noll och modellen blir för enkel. Den låga prestandan i detta intervall kan därför tolkas som ett tecken på underanpassning, då modellen inte fullt ut fångar de underliggande mönstren i data. När C närmar sig 1 kan modellen i större utsträckning utnyttja de informativa termerna, vilket resulterar i en tydlig förbättring i prestanda. Att ökningen inträffar tidigare och att den initiala makro F1-poängen är generellt högre vid mer omfattande dataförbehandling tyder på att dessa steg reducerar brus och förbättrar representationens kvalitet. Detta gör att modellen snabbare når en nivå där relevanta mönster kan utnyttjas effektivt. För högre värden på C (t.ex. $C > 3$) har modellen i praktiken redan nått en stabil lösning, och ytterligare frihet ger ingen förbättring.

För BoW är situationen annorlunda, vilket kan kopplas till dess högdimensionella och brusiga egenskaper. Vid låga värden på C bidrar den starka regulariseringen till att dämpa vikten för mindre informativa eller frekventa termer, vilket förbättrar prestandan. När C ökar får modellen större frihet och börjar i högre grad anpassa sig till brus i data, vilket kan tolkas som en indikation på överanpassning och leder till försämrad eller endast marginell förändring i prestanda. Precis som för TF-IDF, så varierar känsligheten för C mellan förbehandlingsstegen. Vid mer omfattande förbehandling reduceras vokabulärens storlek och brus, vilket gör representationen mer kompakt och minskar modellens känslighet för regularisering. Prestandan blir därmed stabilare över olika värden på C , även om den maximala makro F1-poängen förändras relativt lite.

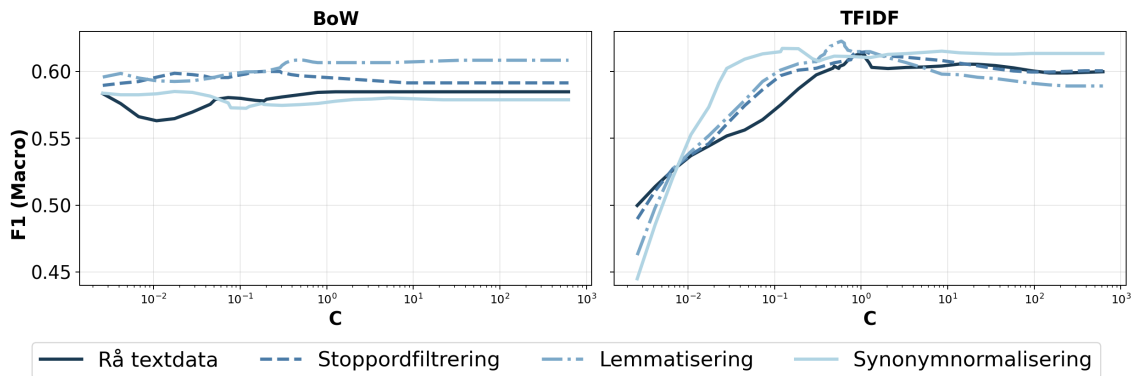


Figur 5: Klassvis noggrannhet för logistisk regression för olika förbehandlingssteg (vänster BoW, höger TF-IDF).

Figur 5 kompletterar denna analys genom att visa den klassvisa noggrannheten. Precis som i fallet med Naiv Bayes uppvisar BoW en mer obalanserad klassificering, medan TF-IDF ger en jämnare prestation mellan klasserna. Eftersom logistisk regression lär vikter för varje vektorkomponent får högfrekventa termer större inflytande vid BoW, medan TF-IDF framhäver mer särskiljande termer och därmed ger mer balanserade prediktioner.

4.4 Stödvektormaskin

I detta avsnitt analyseras hur regulariseringsparametern C påverkar prestandan för stödvektormaskinen (SVM) i kombination med de olika textrepresentationerna och dataförbehandlingarna.

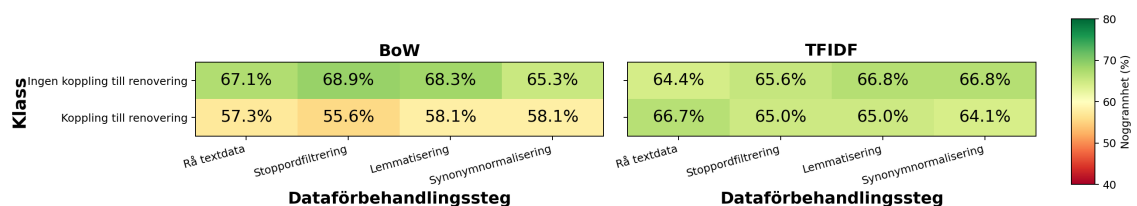


Figur 6: Makro F1-poäng som funktion av C för SVM (vänster BoW, höger TF-IDF).

Figur 6 illustrerar hur makro F1-poängen varierar med regulariseringsparametern C . För BoW-representationen uppvisar modellen en relativt stabil prestanda i intervallet $0,001 \leq C \leq 1$. Vid $C \geq 1$ stabiliseras resultaten ytterligare. Lemmatiseringen uppnår högst makro F1-poäng på 0,61 och håller generellt en högre nivå för alla C -värden jämfört med de andra textrepresentationerna. För TF-IDF-representationen sker en snabb prestandaökning i intervallet $0,001 \leq C \leq 0,1$. När $C \approx 1$ planar kurvorna ut och prestandan blir relativt jämn mellan och inom dataförbehandlingarna. I likhet med BoW är det lemmatiseringen som uppnår den högsta enskilda mätpunkten på makro F1-poäng = 0,62. Men uppvisar en generellt högre prestanda över hela intervallet.

Dessa resultat kan förklaras med att SVM maximerar marginalen mellan klasserna med ett hyperplan. Parametern C ger en avvägning mellan marginalens bredd och antal tillåtna felklassificeringar. BoW är högdimensionell och gles, då den endast beror av termfrekvenser. Vid låga värden på C prioriterar modellen en stor marginal, vilket i kombination med BoW:s brusighet leder till en instabil beslutsgräns då termer förekommer i båda klasser. När värdet på C stiger, tvingas modellen att klassificera korrekt då marginalen är mindre, vilket leder till en stabilisering.

I fallet TF-IDF har informativa termer viktats upp medan högfrekventa termer tonats ned. Detta skapar en tydligare separation mellan klasserna. Den branta ökningen vid låga värden på C i fallet TF-IDF indikerar att modellen är underanpassad då den tillåter för stor marginal. Men den hittar snabbt ett optimalt hyperplan då värdet på C ökar.



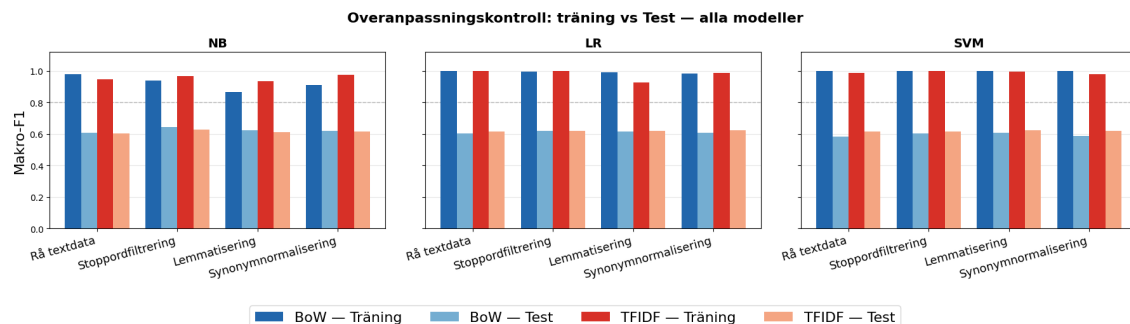
Figur 7: Klassvis noggrannhet för SVM för olika förbehandlingssteg (vänster BoW, höger TF-IDF).

Figur 7 visar den klassvisa noggrannheten för SVM vid de olika förbehandlingsstegen. BoW visar en ojämn prestanda mellan klasserna för alla förbehandlingssteg, medan TF-IDF visar en jämn prestanda för båda klasser. Detta beror sannolikt på att i BoW kan högfrekventa, mindre informativa termer dominera viktinläringen vilket gör att SVM får svårt att skilja på klasserna. TF-IDF motverkar detta genom normalisering, vilket gör att den bättre fångar upp relevanta termer.

4.5 Övergripande analys inklusive generaliseringsförmåga

I detta avsnitt ges en övergripande analys av resultaten med fokus på modellernas generaliseringsförmåga och risk för överanpassning. Vi sammanfattar även hur dataförbehandling, textre-

presentation och modellval påverkar resultaten. Avsnittet belyser dessutom centrala osäkerheter och begränsningar som kan ha påverkat resultaten.



Figur 8: Makro F1-poäng för tränings- och testdata för respektive modell, textrepresentation och förbehandling.

Figur 8 visar makro F1-poäng för tränings- och testdata för varje kombination av dataförbehandling, textrepresentation och modell. Prestandan på testdata har diskuterats i tidigare avsnitt, men presenteras här tillsammans med prestandan på träningsdata för att möjliggöra en analys av överanpassning. För samtliga kombinationer ligger makro F1-poängen för testdata omkring 0,61. Motsvarande värden för träningsdata ligger i intervallet 0,85–1,00. Fullständiga resultat för testdata samt tillhörande standardavvikelser, beräknade över korsvalideringens olika uppdelningar, redovisas i Appendix F.

Skillnaden mellan tränings- och testprestanda indikerar överanpassning, där modellerna i hög grad anpassar sig till specifika mönster eller brus i träningsdata snarare än att lära sig robusta och generaliserbara textmönster. Detta begränsar modellernas generaliseringsförmåga, vilket återspeglas i att samtliga modeller uppvisar en betydligt lägre prestanda på testdata än på träningsdata. Resultatet är i linje med den teoretiska bakgrunden som beskrivs i avsnitten 2.1.1 och 2.2.1, där små och brusiga datamängder förväntas medföra ökad risk för överanpassning.

Vid en övergripande jämförelse framgår det att klassbalansering i träningsdata har störst inverkan på modellernas beteende. Balansering leder till en jämnare klassificering genom förbättrad prestanda för minoritetsklassen. Även valet av textrepresentation spelar en viktig roll. TF-IDF ger generellt en mer balanserad klassificering än BoW, vilket tyder på att textrepresentationens egenskaper har större påverkan på resultaten än valet av klassificeringsmodell. Däremot är effekten av graden av förbehandling relativt begränsad jämfört med effekterna av urval av träningsdata och textrepresentation. Förbehandlingen påverkar främst modellernas känslighet för variationer i hyperparametrar.

Trots skillnader i modellernas egenskaper uppnår samtliga modeller en liknande maximal makro F1-poäng kring 0,61. Skillnaderna mellan modellerna är små i förhållande till den variation som observeras mellan olika uppdelningar av data i korsvalideringen, vilket gör det svårt att avgöra om någon modell faktiskt presterar bättre än de övriga. Resultaten tyder istället på att det kan finnas en övre gräns för den maximala prestanda som kan uppnås givet den aktuella datamängden och de använda textrepresentationerna. Ytterligare en möjlig förklaring till den begränsade prestandan är att den manuella klassificeringen av dokumenten kan innehålla vissa osäkerheter, eftersom annoteringen delvis bygger på subjektiva tolkningar. Exempelvis kan en person ha flyttat delvis på grund av renovering och därför klassificerats som *koppling till renovering*, samtidigt som intervjun också innehåller många andra flyttorsaker som egentligen passar in under *ingen koppling till renovering*. Detta kan göra det svårare för modellerna att hitta specifika mönster inom klasserna. Även datastrukturen i sig utgör en central begränsning. Dokumentens längd och struktur varierar kraftigt; vissa dokument består av dialogbaserade intervjuer medan andra är sammanfattningar. Denna variation kan försämra modellernas förmåga att identifiera konsistenta och generaliserbara mönster. Resultaten kan även ha påverkats av de språkspecifika resurser som användes vid förbehandlingen.

Exempelvis bygger stoppordsfiltreringen på stoppordlistan i Python-biblioteket *NLTK*. Sådana resurser är ofta mindre utvecklade för svenska än för engelska, vilket kan ha begränsat effekten av de använda förbehandlingsmetoderna.

Sammanfattningsvis indikerar resultaten att ytterligare prestandaförbättringar sannolikt kräver förbättringar i datakvalitet och datamängd, bättre förbehandlingsmetoder samt mer avancerade representationsmetoder, snarare än enbart optimering av modeller och hyperparametrar.

5 Slutsatser

I detta avsnitt sammanfattas studiens slutsatser utifrån analysen i kapitel 4. Resultaten visar att samtliga undersökta klassificeringsmodeller uppnår liknande prestanda, att överanpassning sker och att textrepresentation samt datakvalitet har större påverkan på resultaten än valet av klassificeringsmodell. Slutsatser kopplade till studiens specifika delproblem sammanfattas nedan:

Hur bör modellernas prestanda mätas? I vår studie har makro F1-poäng speglat det vi anser vara bästa resultat, det vill säga en jämn klassvis noggrannhet. Eftersom vi gör en binär klassificering, med stor skillnad i datamängd mellan klasserna, behöver vi ta hänsyn till att undvika mått som visar en hög prestanda då den ena klassen dominerar.

Hur påverkar olika dataförbehandlingssteg klassificeringsprestandan hos klassificeringsmodeller? Vår undersökning visar att maximala prestandan för rå textdata var något lägre jämfört med mer förbehandlad data, dock är skillnaden inte tillräckligt stor för att anses betydande. En iakttagelse är dock att olika dataförbehandlingssteg är olika robusta mot förändringar i modellernas hyperparametrar.

Hur påverkar olika textrepresentationer klassificeringsprestandan hos klassificeringsmodeller? Alla kombinationer av modeller och textrepresentationer har uppnått liknande prestanda. Skillnader ligger mer i beteende vid förändringar i modellernas hyperparametrar samt i prediktionsfördelning mellan klasser, där TF-IDF ger jämnare klassvis noggrannhet än BoW.

Hur påverkar olika urval av träningsmängd klassificeringsprestandan hos klassificeringsmodeller? Vårt resultat visar att balanserade träningsmängder är avgörande för att uppnå jämn prestanda mellan klasser. För obalanserade träningsmängder kan majoritetsklassen dominera.

Vilken klassificeringsprestanda kan modellerna Naiv Bayes, logistisk regression och SVM uppnå på en begränsad mängd träningsdata då klassificering ska ske i binära klasser? Naiv Bayes uppnår max makro F1-poäng 0,64 med BoW på stoppordsfiltrering. Logistisk regression uppnår max makro F1-poäng 0,62 med TF-IDF på synonymnormalisering. SVM uppnår max makro F1-poäng 0,62 med TF-IDF på lemmatisering. Fullständiga resultat för högst makro F1-poäng för alla modellkombinationer återfinns i Appendix F.

Hur presterar de framtagna modellerna jämfört med varandra? Alla modeller uppvisade liknande prestanda, men skillnader observerades i hur modellerna påverkades av olika textrepresentationer samt i deras prediktioner. En mer detaljerad analys av modellsensitivitet samt av modellernas överensstämmelse med varandra återfinns i appendix G respektive appendix H.

Ett förslag till framtida kunskapsinsamling är fler fallstudier med olika kombinationer av datamängder och datakvalitet. Så att organisationer som vill ta fram egna modeller genom träning på egen data kan få en uppfattning om möjliga prestandanivåer, också givet aspekten datakvalitet. För att få en bättre uppfattning om vad som utgör tillräckligt god prestanda, skulle det också vara intressant att sätta modellernas prestanda i relation till i hur hög utsträckning två människor klassificerar texter på samma sätt, vilket gjordes i den studie som presenteras i Appendix C.

Referenser

- Altman, N., & Krzywinski, M. (2018). The curse(s) of dimensionality. *Nature Methods*, 15(6), 397–400. <https://doi.org/10.1038/s41592-018-0019-x>
- Baladram, S. (2024, 26. oktober). Oversampling and Undersampling, Explained: A Visual Guide with Mini 2D Dataset. Hämtad 15 april 2026, från <https://towardsdatascience.com/oversampling-and-undersampling-explained-a-visual-guide-with-mini-2d-dataset-1155577d3091/>
- Blog, N. D. (2022). Faster Text Classification with Naive Bayes and GPUs. Hämtad 10 maj 2026, från <https://developer.nvidia.com/blog/faster-text-classification-with-naive-bayes-and-gpus/>
- Bonaccorso, G. (2018). *Machine Learning Algorithms: Popular algorithms for data science and machine learning* (2. utg.). Packt Publishing.
- Bouhsine, I., m. fl. (2026). In Defense of Cosine Similarity: The Expressive Power of Embeddings with ℓ_2 -Normalization. *arXiv preprint arXiv:2602.19393*. <https://arxiv.org/abs/2602.19393>
- Chiera, B. A., & Korolkiewicz, M. W. (2017). Visualizing Big Data: Everything Old Is New Again. I F. P. García Márquez & B. Lev (Red.), *Big Data Management* (s. 1–27). Springer eBooks.
- Cui, Z. (2021). Machine Learning and Small Data. *Educational Measurement: Issues and Practice*, 40(4), 8–12. <https://doi.org/https://doi.org/10.1111/emip.12472>
- Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. <https://doi.org/10.1017/9781108679930>
- Dewi, C., Chen, R.-C., Christanto, H. J., & Cauteruccio, F. (2023). Multinomial Naïve Bayes Classifier for Sentiment Analysis of Internet Movie Database. *Vietnam Journal of Computer Science*, 10(4), 485–498. <https://doi.org/10.1142/S2196888823500100>
- Grandini, M., Bagli, E., and Visani, G. (2020, 14. augusti). Metrics for Multi-Class Classification: an Overview. Hämtad 31 mars 2026, från <https://arxiv.org/abs/2008.05756>
- Gutman, A. J., & Goldmeier, J. (2021). *Becoming a Data Head - How to Think, Speak, and Understand Data Science, Statistics, and Machine Learning*. John Wiley & Sons.
- He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*. Hämtad 22 mars 2026, från <https://doi.org/10.1109/TKDE.2008.239>
- Jurafsky, D., & Martin, J. H. (2026a). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition [Draft of the 3rd edition, Stanford University]. https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf
- Jurafsky, D., & Martin, J. H. (2026b). Speech and Language Processing: Appendix B – Naive Bayes. <https://web.stanford.edu/~jurafsky/slp3/B.pdf>
- Kinoshita, E., & Mizuno, T. (2017). What Is Big Data. I F. P. García Márquez & B. Lev (Red.), *Big Data Management* (s. 91–101). Springer eBooks.
- Kokol, P., Kokol, M., & Zagoranski, S. (2022). Machine learning on small size samples: A synthetic knowledge synthesis. *Science Progress*, 105(1), 1–16. <https://doi.org/10.1177/00368504211029777>
- Lenovo. (u.å.). Understanding Underfitting in Machine Learning. Hämtad 23 april 2026, från <https://www.lenovo.com/us/en/knowledgebase/understanding-underfitting-in-machine-learning/>
- Lindholm, A., Wahlström, N., Lindsten, F., & Schön, T. B. (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press. <https://doi.org/10.1017/9781108919371>
- Lodhi, S. S., Kumar, N., & Pandey, P. K. (2023). Autonomous vehicular overtaking maneuver: A survey and taxonomy. *Vehicular Communications*. <https://doi.org/10.1016/j.vehcom.2023.100553>
- Mahesh, T. R., m. fl. (2023). The stratified k-folds cross-validation and class-balancing to solve class imbalance problem. *MethodsX*, 11, 102324. <https://doi.org/10.1016/j.mex.2023.102324>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

- Manning, C. D., Raghavan, P., & Schütze, H. (2009). *Introduction to Information Retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>
- Naser, M. Z. (2026). A review of machine learning with small and limited data. *Journal of Big Data*, 13, Article 18. <https://doi.org/10.1186/s40537-025-01346-9>
- Opitz, J., & Burst, S. (2019, 8. november). Macro F1 and Macro F1. Hämtad 10 maj 2026, från <https://doi.org/10.48550/arXiv.1911.03347>
- OxfordEnglishDictionary. (u. å-a). computational linguistics. I *OxfordEnglishDictionary*. Hämtad 5 maj 2026, från https://www.oed.com/dictionary/computational-linguistics_n?tab=meaning_and_use#119677436100
- OxfordEnglishDictionary. (u. å-b). natural language processing. I *OxfordEnglishDictionary*. Hämtad 5 maj 2026, från https://www.oed.com/dictionary/natural-language-processing_n?tab=meaning_and_use#11230004100
- Padurariu, C., & Breaban, M. E. (2019). Dealing with Data Imbalance in Text Classification. *Procedia Computer Science*, 159, 736–745. <https://doi.org/10.1016/j.procs.2019.09.229>
- Ramanathan, T. (u. å). natural language processing. I *Britannica Academic*. Hämtad 5 maj 2026, från <https://academic-eb-com.eu1.proxy.openathens.net/levels/collegiate/article/natural-language-processing/636044>
- scikit-learn developers. (2026a). Feature extraction. Hämtad 26 mars 2026, från https://scikit-learn.org/stable/modules/feature_extraction.html
- scikit-learn developers. (2026b). LogisticRegression. Hämtad 13 april 2026, från https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- scikit-learn developers. (2026c). Naive Bayes. Hämtad 25 mars 2026, från https://scikit-learn.org/stable/modules/naive_bayes.html
- scikit-learn developers. (2026d). naive_bayes.py. Hämtad 3 maj 2026, från https://github.com/scikit-learn/scikit-learn/blob/fe2edb3cd/sklearn/naive_bayes.py#L811
- scikit-learn developers. (2026e). TfidfTransformer. Hämtad 1 maj 2026, från https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html
- Siino, M., Tinnirello, I., & La Cascia, M. (2023). Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on Transformers and traditional classifiers. *Information Systems*, 121, 102342. <https://doi.org/10.1016/j.is.2023.102342>
- Stanford University. (n.d.). CS231n: Linear Classification. Hämtad 12 maj 2026, från <https://cs231n.github.io/linear-classify/>
- Zhang, W., & Gao, F. (2011). An Improvement to Naive Bayes for Text Classification. *Procedia Engineering*, 15, 2160–2164. <https://doi.org/10.1016/j.proeng.2011.08.404>

Appendix

A Användning av AI

Generativ AI i form av stora språkmodeller (LLM), såsom OpenAIs ChatGPT och GitHub Copilot, har använts som hjälp vid kodning, grammatikkorrigerering och förbättring av formuleringar samt för att skriva matematiska formler och tabeller i LaTeX. AI-verktygen har även använts som komplement för att bygga upp en initial förståelse för olika modeller.

B Teori: Maskininlärning

Maskininlärning handlar om att utforma modeller som kan utvinna information som är av värde från data (Deisenroth m. fl., 2020, s. 3). Det kan till exempel handla om att utifrån data om fotbollsspelares positioner på planen, när de lyckas respektive inte lyckas skjuta i mål, försöka förutse sannolikheten att det blir mål när man skjuter från vilken position på planen som helst (Lindholm m. fl., 2022, s. 7–8). Det skulle också kunna handla om att utifrån data om luftföroreningar vid olika tidpunkter och olika platser försöka förutse luftföroreningar på olika platser vid framtida tidpunkter (Lindholm m. fl., 2022, s. 9–10). Målet är att utforma en modell som kan göra förutsägelser om situationer/data som den inte tidigare har sett (Deisenroth m. fl., 2020, s. 6).

Modellerna utformas mer specifikt genom att deras parametrar optimeras utifrån faktiska data, det är därför man talar om inlärning (Deisenroth m. fl., 2020, s. 3). Man kan alltså säga att modellen lär sig hur den ska se ut för att kunna göra korrekta förutsägelser om en viss typ av data.

I den här rapporten definieras begreppet ”modell” enligt Deisenroth m. fl. (2020, 3, egen översättning) som ”ett system som gör förutsägelser baserat på inputdata”. Författarna tar upp att ibland så kan man med ”modell” också mena själva systemet som justerar parametrarna vid träning (Deisenroth m. fl., 2020, s. 4), men det är alltså inte vad som menas i denna rapport.

Det finns olika typer av modeller. Deisenroth m. fl. (2020, s. 228) har perspektivet att modeller kan vara funktioner eller probabilistiska modeller. De menar att om en modell är en funktion, så ger den för viss indata viss utdata och om en modell är probabilistisk, så beskriver den möjliga funktioner i form av en fördelning (Deisenroth m. fl., 2020, s. 229). Man kan också skilja på regression och klassificering (Lindholm m. fl., 2022, s. 15). I det förstnämnda fallet är förutsägelsen numerisk, vilket innebär naturlig talföljd och det kan även innebära kontinuerliga värden. Det sistnämnda innebär att förutsägelsen är kategorisk. För att förtydliga kan man säga att ett klassificeringsproblem handlar om att kunna förutsäga en särskild klass och där det bara finns ett ändligt antal sådana klasser (Lindholm m. fl., 2022, s. 4).

Det finns också olika typer av data och maskininlärning kan appliceras på t.ex. siffror, text eller bilder (Deisenroth m. fl., 2020, s. 226). Datorlingvistik (computational linguistics) är ett ”område inom lingvistik där tekniker från datavetenskapen används för analys och syntes av språk och tal” (OxfordEnglishDictionary, u. å-a, egen översättning). En form av datorlingvistik är den så kallade naturliga språkbehandlingen (natural language processing, NLP), där ”texter på naturligt språk processas av datorer” (OxfordEnglishDictionary, u. å-b, egen översättning). Inom NLP används maskininlärning (Ramanathan, u. å). Deisenroth m. fl. (2020, s. 227) beskriver att inputdata vid maskininlärningen har formen av vektorer med en viss dimension, där dimensionen bestäms av så kallade attribut (features). Om en indatapunkt representerar en person, så kan till exempel attribut vara kön och ålder.

Det går också att träna modeller på olika sätt. Två huvudvarianter är övervakad (supervised) och oövervakad (unsupervised) maskininlärning. Vid övervakad maskininlärning innehåller träningsdata par av indatavärden och utdatavärden (Lindholm m. fl., 2022, s. 13), träningsdata är annoterad eller etiketterad (labeled) (Lindholm m. fl., 2022, s. 14). Man kan beskriva det som att det finns ett facit för ett visst antal exempel, som modellen alltså kan optimeras utifrån. Vid oövervakad

maskininlärning har inte träningsdata några etiketter, utan det handlar om att modellen behöver förstå egenskaper hos data ändå (Lindholm m. fl., 2022, s. 259). Faktum är att såväl regression som klassificering, vilket är de de typer av maskininlärningsproblem som behandlas i denna rapport, är undertyper av övervakad maskininlärning (Lindholm m. fl., 2022, s. 14). Ett exempel på ett oövervakat maskininlärningsproblem är istället klustring (clustering), vilket handlar om att hitta grupper av data som liknar varandra och om att dela in dessa i diskreta kluster (Lindholm m. fl., 2022, s. 259).

C Teori: Studie av prestanda vid begränsade datamängder

Det finns forskning som har mätt prestandan för modeller som har tränats på begränsade datamängder. Exempelvis har Cui (2021) jämfört prestandan för modellerna multinomial Naiv Bayes och stödvektormaskin (SVM) vid övervakad maskininlärning och binär klassificering. Studien undersökte om maskininlärningsmodeller, tränade på en begränsad mängd träningsdata, skulle kunna mäta sig med mänskliga experter när det kommer till att gå igenom ett slags avvikelserapporter vid genomförande av standardiserade testsituationer inom utbildningsområdet. Den här genomgången är vid många avvikelser tidskrävande för människor.

Träningsmängden i den här studien utgjordes mer specifikt av 1.500 kommentarer och testmängden utgjordes av 644 kommentarer, vilket vi tolkar som kommentarer om avvikelser i avvikelserapporterna. Data bestod alltså av text och texten delades upp i ord, textrepresentationen BoW användes, stoppord togs bort, olika böjningar av samma ord behandlades som samma ord (som exempel ges att orden kopia, kopiera och kopierad behandlades som samma ord) och de 3000 vanligaste orden användes som attribut (features). Implementering skedde med Scikit-learn-paket (package).

Resultatet visade att vid utvärdering på testdata, så var det inte så stor skillnad mellan de olika typerna av modeller. För multinomial Naiv Bayes uppnåddes för de prestandamått som även används i denna rapport: noggrannhet 0,86; precision 0,70; känslighet 0,74 och F1 0,72. För stödvektormaskin uppnåddes: noggrannhet 0,86; precision 0,68; känslighet 0,81 och F1 0,74. I studien (Cui, 2021) användes dock även modellerna på ytterligare data, från standardiserade utbildningstest som genomförts ett år senare. Då blev resultaten istället för multinomial Naiv Bayes: noggrannhet 0,84; precision 0,61; känslighet 0,54 och F1 0,58. För stödvektormaskin blev resultaten: noggrannhet 0,76; precision 0,43; känslighet 0,55 och F1 0,48.

Cui (2021) drar slutsatsen att multinomial Naiv Bayes är ett bättre alternativ för den aktuella tillämpningen. Det framkommer att 84 procent av bedömningarna som multinomial Naiv Bayes gjorde var desamma som de som en mänsklig expert hade gjort, vilket är i samma storleksordning som skillnaden mellan två mänskliga experter. Författaren drar slutsatsen att det inte spelar någon roll om avvikelserapporterna går igenom av mänskliga experter eller av maskininlärningsmodellen, även om modellen bara tränats på 1.500 kommentarer.

Det bör poängteras att Cui (2021) studie inte är helt jämförbar med vår studie som presenteras i denna rapport, men den ger ändå en indikation om att det kan vara möjligt att uppnå tillräckligt goda resultat även om de datamängder som maskininlärningsmodeller tränas på inte är särskilt stora.

D Teori: Prestandamått

Några etablerade prestandamått som används för utvärdering av klassificeringsmodeller är noggrannhet (accuracy), precision, känslighet/klassvis noggrannhet (recall) och F1-poäng (F1-score) (Jurafsky & Martin, 2026a).

D.1 Noggrannhet

Noggrannhet är sannolikheten att en modells klassificering av ett dokument är korrekt (Grandini, M., Bagli, E., and Visani, G., 2020) och beräknas enligt

$$\text{Noggrannhet} = \frac{\sum_{i=1}^n \text{TP}_i}{m}$$

Där TP_i representerar de fall där modellen har klassificerat rätt för klass i , n är antal klasser och m är antal dokument totalt tillhörande alla klasser. Då noggrannhet ensamt kan ge en missvisande bild vid obalanserade klasser används ofta kompletterande prestandamått (He & Garcia, 2009).

D.2 Precision

Precision är ett mått på hur exakt modellen är vid klassificering (He & Garcia, 2009). Mer specifikt är det ett mått på hur många av dokumenten som klassificerats till klass i som är korrekt klassificerade. Detta beräknas enligt

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}$$

Där TP_i representerar de fall där klassificeringen blivit korrekt för klass i som kontrolleras, medan FP_i representerar de fall där klassificeringen har tilldelat ett dokument klass i men dokumentet tillhör egentligen en annan klass.

För att kolla på modellens precision generellt över alla klasser kan makroprecision användas där precisionen för varje enskild klass summeras och divideras med antal klasser (n) enligt:

$$\text{Precision}_{\text{makro}} = \frac{\sum_{i=1}^n \text{Precision}_i}{n}$$

Ett makrogenomsnitt innebär att varje klass bidrar med samma vikt till resultatet, man tar alltså inte hänsyn till storleken på klassen (Grandini, M., Bagli, E., and Visani, G., 2020). Här anses modellens förmåga att klassificera mindre kategorier lika viktig som för större klasser.

D.3 Känslighet/Klassvis noggrannhet

Känslighet är ett mått på modellens fullständighet och beskriver hur stor andel av dokumenten i klass i som blivit korrekt klassificerade (He & Garcia, 2009). I denna rapport används även termen klassvis noggrannhet synonymt med känslighet, syftet är att underlätta läsning och tolkning av resultat. Måttet beräknas enligt:

$$\text{Känslighet}_i = \frac{\text{TP}_i}{N_i}$$

Där TP_i är antal korrekt klassificeringar för klass i och N_i antalet datamängder som faktiskt tillhör klass i .

Ett genomsnitt av klassernas känslighet kan beräknas enligt makromedelvärde. Då summeras känsligheten för klasserna och divideras med antalet klasser (Grandini, M., Bagli, E., and Visani, G., 2020), enligt:

$$\text{Känslighet}_{\text{makro}} = \frac{\sum_{i=1}^n \text{Känslighet}_i}{n}$$

D.4 F1-poäng

F1-poängen kombinerar resultaten från känslighet och precision och är användbart för att hitta den bästa avvägningen mellan dessa två (Grandini, M., Bagli, E., and Visani, G., 2020). F1-poängen beräknas enligt:

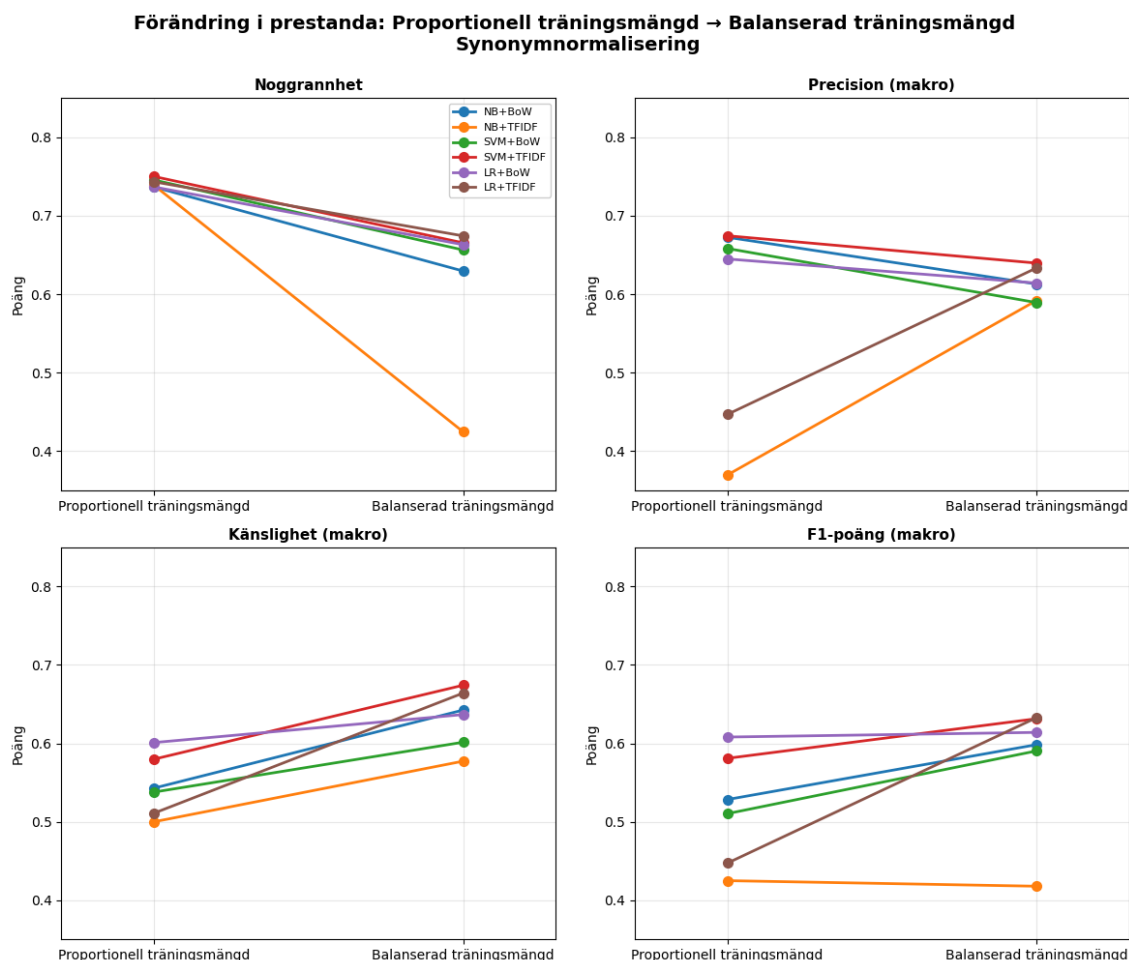
$$\text{F1-poäng}_i = \frac{2 \cdot \text{Känslighet}_i \cdot \text{Precision}_i}{\text{Känslighet}_i + \text{Precision}_i}$$

Som ett sammanfattande mått kan makro F1-poäng användas. Detta mått är lämpligt att använda vid obalanserade dataset eftersom det behandlar alla klasser som lika viktiga (Opitz & Burst, 2019). Makro F1-poäng beräknas genom medelvärdet av samtliga n klassers F1-poäng enligt:

$$\text{F1-poäng}_{\text{makro}} = \frac{\sum_{i=1}^n \text{F1-poäng}_i}{n}$$

E Resultat och diskussion: Val av prestandamått

För att välja vilket eller vilka prestandamått som borde användas så jämfördes alla modell- och textrepresentationskombinationer (hädanefter kallade modellkombinationer) vid olika träningsmängder för de olika prestandamåtten.



Figur 9: Prestandamåtten för modellkombinationerna vid olika stor träningsmängd vid synonymnormalisering.

I figur 9 visas en överblick över hur de olika prestandamåtten noggrannhet, makroprecision, makrokänslighet och makro F1-poäng påverkas av urvalet av träningsdata. De två träningsmängder som har jämförts är proportionell träningsmängd med 80% av all data ur varje klass och balanserad träningsmängd med 93 slumpmässiga dokument ur varje klass.

När träningsmängden för majoritetsklassen minskar, så minskar även noggrannheten för alla modellkombinationer (se övre vänstra hörnet). Detta beror på att för obalanserad träningsmängd så var modellkombinationen bra på att klassificera rätt för majoritetsklassen, men för minoritetsklassen var antal rätt klassificeringar väldigt få. När träningsmängden balanserades ökade antal rätt klassificeringar i minoritetsklassen markant och för majoritetsklassen minskade det något. Vid analys av resultaten värdesätter vi att modellerna kan klassificera båda klasser. Att detta inte representeras i noggrannhetsmättet är ett vanligt problem och vi har därför valt att fokusera på de andra prestandamåtten då de bättre speglar det vi är intresserade av. Däremot har vi valt att i vissa fall kolla på klassvis noggrannhet då det tydligt visar hur prestandan hos de olika klasserna var för sig ändras. Med klassvis noggrannhet, även kallat känslighet, avses i denna rapport modellens förmåga att korrekt identifiera instanser av en specifik klass.

För makroprecision (se övre högra hörnet) kan man se att den minskar för de flesta modellkombinationerna då träningsmängden är balanserad medan det blir betydligt bättre för logistisk regression i kombination med TF-IDF och Naiv Bayes i kombination med TF-IDF. Att resultatet förbättras för båda dessa beror på att när träningsmängden var obalanserad så klassificerade dessa modeller nästan all data att tillhöra majoritetsklassen (se figur 1), vilket innebär att precisionen för minoritetsklassen var noll eller nära noll och drog ner snittet för modellkombinationerna. Medan när träningsmängden balanserades blev modellkombinationerna bättre på att klassificera minoritetsklassen och presterade då på samma nivå som de andra modellkombinationerna som hade en minskning i majoritetsklassen och en stor ökning för minoritetsklassen.

Makrokänslighet ökar för varje modellkombination då träningsmängden är balanserad jämfört med när den är obalanserad. Detta är rimligt då modellkombinationerna presterade betydligt sämre på den mindre klassen då träningsmängden var proportionell vilket drar ner snittet för de båda klasserna. När modellkombinationerna sedan kan klassificera båda klasserna väl blir den totala känsligheten bättre.

Makro F1-poängen ökar vid balansering av träningsmängden, förutom gällande Naiv Bayes i kombination med TF-IDF och logistisk regression i kombination med BoW som är nästintill oförändrade (se nedre högra hörnet). Anledningen till att Naiv Bayes med TF-IDF uppvisar en jämn och låg nivå är att modellkombinationen vid obalanserade träningsmängder klassificerade samtliga dokument som majoritetsklassen. När träningsmängden balanserades klassificerade modellkombinationen i stället nästan all data som minoritetsklassen (se figur 1). Att logistisk regression i kombination med BoW håller en jämn nivå beror på att denna kombination vid obalanserad data var betydligt bättre på att klassificera i båda klasser jämfört med de andra modellkombinationerna, alltså blir skillnaderna inte lika stora då träningsmängden balanserades.

F1-poäng är en kombination av känslighet och precision. Trots att precisionen minskar för de flesta modellkombinationerna, vid övergång till balanserad data, så ökar makro F1-poängen då känsligheten ökar tillräckligt mycket för att väga upp för minskningen i precision. Därför är det makro F1-poängen vi valt att huvudsakligen studera, då det speglar den förbättring i resultat som vi ser men att den också kan fånga upp de fall där ingen större förbättring eller försämring sker. Det är Naiv bayes i kombination med TF-IDF och logistisk regression i kombination med BoW som visar på ett nästan oförändrat resultat.

F Resultat och diskussion: Hyperparametrar med högst makro F1-poäng

Tabellerna nedan presenterar de hyperparameterkombinationer som gav högst makro F1-poäng för varje kombination av dataförbehandling och textrepresentation, samt vid balanserad träningsmängd, inom respektive modell.

Huvudsyftet med studien var att undersöka modellerna Naiv Bayes, logistisk regression och SVM. Därför har deras hyperparametrars inverkan analyserats i kapitel 4. Olika värden för min_df, max_df och ngram har endast testats för att hitta bästa möjliga prestanda för modellerna.

Tabell 5: Hyperparameterkombinationer med högst makro F1-poäng för Naiv Bayes

Textrepr.	Förbehandling	Hyperparameter	min_df	max_df	ngram	makro F1-poäng
BoW						
BoW	RAW	$\alpha = 0,091030$	1	0,8	(1,1)	$0,6093 \pm 0,0503$
BoW	NO_STOPWORDS	$\alpha = 0,372759$	2	0,9	(1,1)	$0,6425 \pm 0,0714$
BoW	LEMMATIZED	$\alpha = 0,120679$	5	0,8	(1,1)	$0,6256 \pm 0,0643$
BoW	FINAL	$\alpha = 0,091030$	5	1,0	(1,2)	$0,6216 \pm 0,0544$
TF-IDF						
TF-IDF	RAW	$\alpha = 0,004095$	5	0,8	(1,1)	$0,6050 \pm 0,0475$
TF-IDF	NO_STOPWORDS	$\alpha = 0,091030$	4	0,8	(1,1)	$0,6279 \pm 0,0490$
TF-IDF	LEMMATIZED	$\alpha = 0,241030$	5	0,8	(1,1)	$0,6133 \pm 0,0464$
TF-IDF	FINAL	$\alpha = 0,091030$	4	0,8	(1,2)	$0,6144 \pm 0,0406$

Tabell 6: Hyperparameterkombinationer med högst makro F1-poäng för logistisk regression

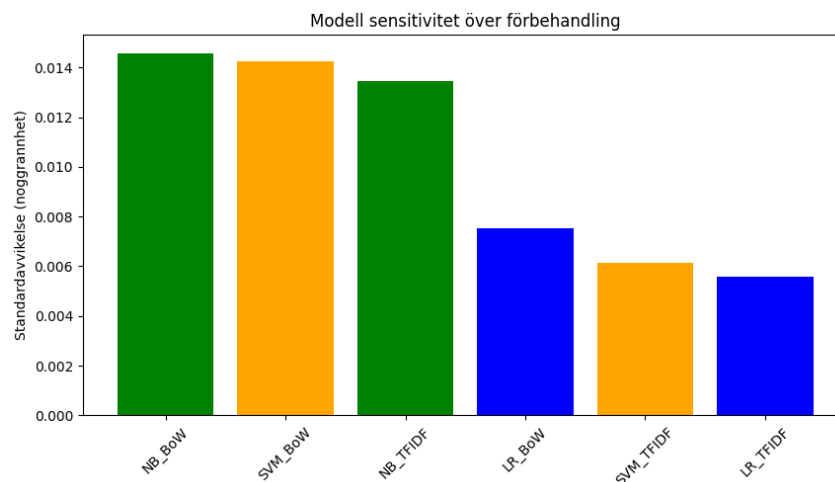
Textrepr.	Förbehandling	Hyperparameter	min_df	max_df	ngram	makro F1-poäng
BoW						
BoW	RAW	$C = 0,655129$	4	0,8	(1,1)	$0,6053 \pm 0,0422$
BoW	NO_STOPWORDS	$C = 0,091030$	3	0,8	(1,1)	$0,6187 \pm 0,0307$
BoW	LEMMATIZED	$C = 0,068665$	3	0,9	(1,2)	$0,6144 \pm 0,0283$
BoW	FINAL	$C = 0,029471$	5	0,9	(1,2)	$0,6082 \pm 0,0360$
TF-IDF						
TF-IDF	RAW	$C = 19,306977$	4	0,8	(1,1)	$0,6165 \pm 0,0226$
TF-IDF	NO_STOPWORDS	$C = 17,376280$	5	0,8	(1,1)	$0,6215 \pm 0,0144$
TF-IDF	LEMMATIZED	$C = 0,932630$	5	1,0	(1,2)	$0,6199 \pm 0,0515$
TF-IDF	FINAL	$C = 1,821231$	2	0,9	(1,2)	$0,6244 \pm 0,0313$

Tabell 7: Hyperparameterkombinationer med högst makro F1-poäng för SVM

Textrepr.	Förbehandling	Hyperparameter	min_df	max_df	ngram	makro F1-poäng
BoW						
BoW	RAW	$C = 13,738238$	1	1,0	(1,2)	$0,5847 \pm 0,0240$
BoW	NO_STOPWORDS	$C = 0,188739$	1	0,8	(1,2)	$0,6023 \pm 0,0563$
BoW	LEMMATIZED	$C = 0,489390$	2	0,8	(1,2)	$0,6084 \pm 0,0508$
BoW	FINAL	$C = 0,017433$	5	0,9	(1,2)	$0,5880 \pm 0,0463$
TF-IDF						
TF-IDF	RAW	$C = 0,945656$	3	0,8	(1,1)	$0,6144 \pm 0,0164$
TF-IDF	NO_STOPWORDS	$C = 1,268961$	2	0,9	(1,1)	$0,6160 \pm 0,0190$
TF-IDF	LEMMATIZED	$C = 0,539390$	3	0,9	(1,2)	$0,6235 \pm 0,0256$
TF-IDF	FINAL	$C = 0,172790$	2	0,9	(1,2)	$0,6206 \pm 0,0323$

G Resultat och diskussion: Modellsensitivitet

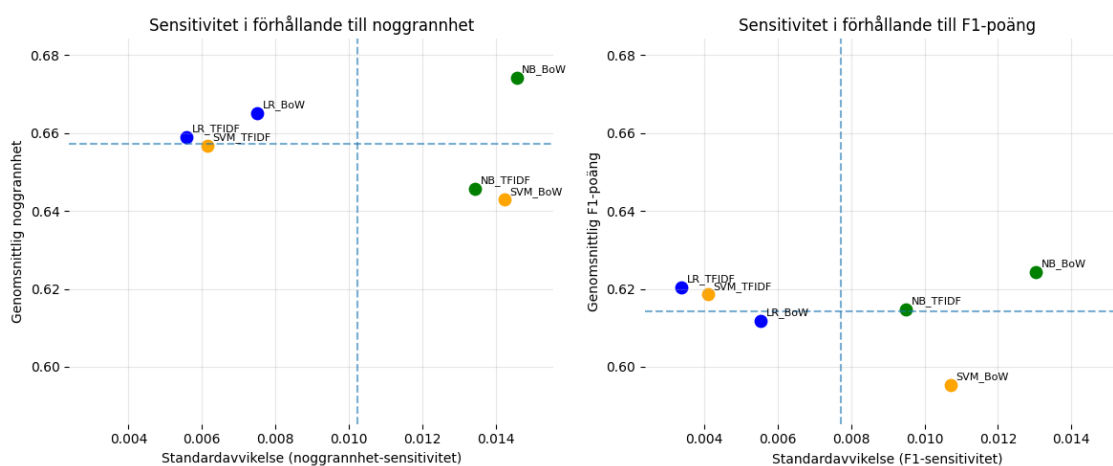
I detta appendix analyseras modell- och textrepresentationskombinationernas (hädanefter kallade modellkombinationer) sensitivitet för förbehandling av text.



Figur 10: Standardavvikelse för noggrannhet med avseende på förbehandling

Figur 10 visar standardavvikelsen i modellkombinationernas noggrannhet med avseende på olika förbehandlingsmetoder. Resultaten visar tydliga skillnader mellan modellkombinationerna. Naiv Bayes med BoW har högst standardavvikelse, vilket indikerar att modellkombinationens prestanda är starkt beroende av hur texten förbehandlas. I kontrast har logistisk regression med TF-IDF den lägsta standardavvikelsen, vilket tyder på att modellkombinationen är stabil och mindre känslig för variationer i förbehandling. Även SVM med TF-IDF uppvisar låg variation, vilket tyder på god robusthet.

Ett tydligt mönster är att TF-IDF-representation generellt leder till lägre sensitivitet jämfört med BoW, särskilt för logistisk regression och SVM. Detta tyder på att TF-IDF bidrar till mer konsekventa och generaliserbara modellresultat.



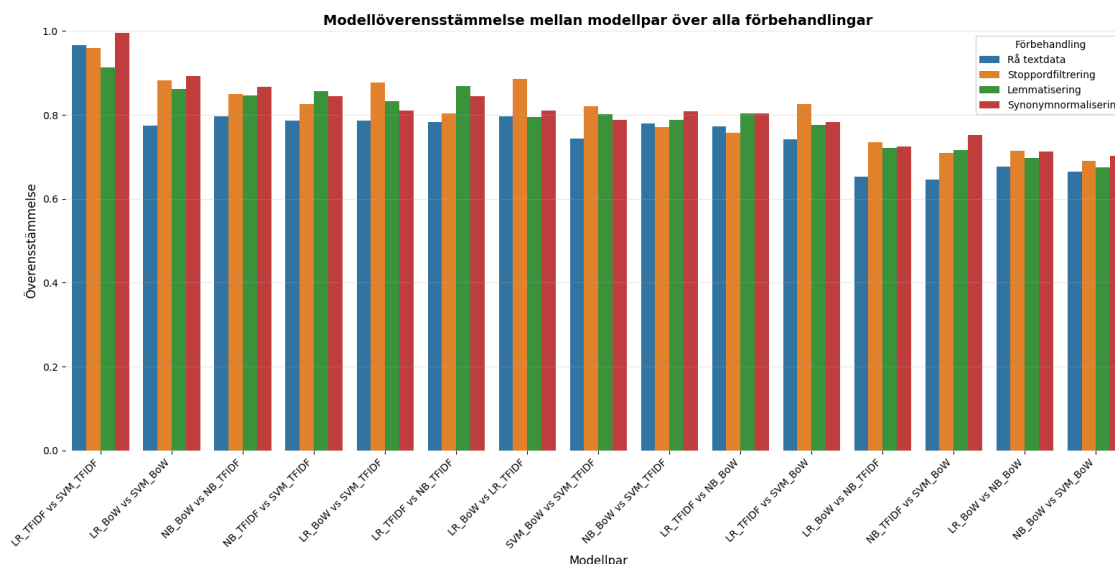
Figur 11: Sensitivitet i förhållande till noggrannhet och F1-poäng

Figur 11 visar modellkombinationernas sensitivitet i förhållande till både noggrannhet och F1-poäng, där de streckade linjerna representerar medelvärden. Logistisk regression hamnar i den vänstra delen av båda graferna, vilket indikerar låg sensitivitet och därmed hög robusthet mot

variationer i förbehandling. För SVM observeras ett tydligt beroende av textrepresentation, där TF-IDF resulterar i lägre sensitivitet och mer stabil prestanda, medan BoW leder till högre variation. Detta tyder på att SVM är mer känslig för valet av representation än logistisk regression.

H Resultat och diskussion: Modellöverensstämmelse

I detta appendix undersöks och analyseras överensstämmelsen mellan modell- och textrepresentationskombinationerna (hädanefter kallade modellkombinationerna).



Figur 12: Parvis modellkombinationsöverensstämmelse med avseende på de olika förbehandlingarna

I figur 12 visas tydliga skillnader i hur modellkombinationerna förhåller sig till varandra. Särskilt syns en hög överensstämmelse mellan logistisk regression och SVM med TF-IDF. Detta kan delvis förklaras av att båda modellerna baserar sina prediktioner på liknande viktade representationer av texten och därmed tenderar att identifiera liknande termer som viktiga för klassificeringen.

Naiv Bayes avviker däremot något, vilket tyder på att den viktar termer annorlunda jämfört med logistisk regression och SVM. En möjlig förklaring är att Naiv Bayes bygger på sannolikhetsbaserade antaganden och antar att termerna är oberoende av varandra, medan logistisk regression och SVM lär sig beslutsgränser genom att tilldela vikter till olika termer. Trots att både Naiv Bayes och logistisk regression är probabilistiska modeller skiljer sig deras underliggande antaganden och beräkningar åt, vilket kan bidra till skillnaderna i deras prediktioner.

Modeller baserade på TF-IDF uppvisar generellt en hög överensstämmelse mellan varandra för de olika förbehandlingsmetoderna. Detta kan bero på att TF-IDF skapar mer informativa och konsekventa representationer av texten genom att minska påverkan från vanligt förekommande termer. Modeller baserade på BoW tenderar däremot att uppvisa en större variation i överensstämmelse. En möjlig förklaring för detta är att textrepresentationen påverkas betydligt mer av termernas absoluta frekvens och därmed skapar större variation mellan modellernas beteenden.

Vidare indikerar resultatet att förbehandlingen även har en påverkan på i vilken grad modellerna uppvisar liknande beteenden. Resultatet tyder på att förbehandling generellt minskar skillnaderna mellan modellkombinationerna, vilket kan förklaras genom att bruset och variationen i texten reduceras. Därav blir data mer enhetlig och enklare för modellerna att tolka på ett likartat sätt. Samtidigt kvarstår vissa variationer, vilket indikerar att modellernas individuella egenskaper inte helt elimineras.

Det är dock viktigt att betona att hög överensstämmelse mellan modellkombinationer kan bero på två olika orsaker. Antingen kan modellerna vara metodmässigt lika och därmed ge liknande prediktioner, eller så kan modellerna uppnå god prestanda och därför klassificera samma data korrekt. För att få en bättre inblick för vad överensstämmelsen mellan modellerna faktiskt innebär

behöver den därför analyseras tillsammans med modellernas prestanda.

Tabell 8: Noggrannhet och F1-poäng för samtliga förbehandlingar, modeller och textrepresentationer.

Förbehandling	Modell	Noggrannhet	F1-poäng
Rå textdata	LR_BoW	0,654	0,607
	LR_TFIDF	0,652	0,616
	NB_BoW	0,656	0,610
	NB_TFIDF	0,627	0,605
	SVM_BoW	0,629	0,585
	SVM_TFIDF	0,650	0,615
Stoppordsfiltrerad	LR_BoW	0,670	0,619
	LR_TFIDF	0,659	0,622
	NB_BoW	0,692	0,641
	NB_TFIDF	0,659	0,628
	SVM_BoW	0,654	0,601
	SVM_TFIDF	0,654	0,616
Lemmatiserad	LR_BoW	0,670	0,614
	LR_TFIDF	0,661	0,619
	NB_BoW	0,674	0,625
	NB_TFIDF	0,645	0,613
	SVM_BoW	0,656	0,607
	SVM_TFIDF	0,663	0,624
Synonymnormaliserad	LR_BoW	0,667	0,608
	LR_TFIDF	0,665	0,624
	NB_BoW	0,674	0,621
	NB_TFIDF	0,652	0,614
	SVM_BoW	0,632	0,587
	SVM_TFIDF	0,661	0,620

Utifrån figur 12 och tabell 8 kan en tendens observeras där modeller med högre genomsnittlig överensstämmelse ofta uppvisar något högre och mer stabil F1-poäng för olika förbehandlingar. Sambandet är dock inte entydigt, eftersom vissa modeller med lägre överensstämmelse fortfarande uppnår jämförbar prestanda eller högre prestanda.

Exempelvis uppvisar logistisk regression och SVM med TF-IDF en hög överensstämmelse samtidigt som de har en relativt medelmåttig noggrannhet och F1-poäng. Detta indikerar att modellerna inte enbart gör liknande prediktioner, utan även tenderar att felklassificera samma data. Samtidigt uppnår Naiv Bayes i flera fall relativt god prestanda trots lägre överensstämmelse med övriga modellkombinationer. Detta antyder att modellen delvis fångar upp andra kompletterande mönster i data.

Sammantaget tyder resultaten på att mer omfattande förbehandling generellt leder till ökad överensstämmelse mellan modellkombinationerna, medan förbättringarna i prestanda är relativt begränsade. Detta antyder att förbehandlingen i högre grad bidrar till mer likartade beteenden hos modellkombinationerna än till tydliga prestandaförbättringar. Resultaten indikerar även att modellkombinationerna delvis kompletterar varandra. Det är därför möjligt att en kombination av flera modellkombinationers prediktioner, där majoritetens prediktion används, skulle kunna bidra till högre prestanda än användningen av en enskild modellkombination. Detta har dock inte undersökts inom ramen för studien.

I Resultat och diskussion: Ord med stor påverkan

I detta appendix visas orden som vardera modell ansåg påverka mest vid klassificering. Ord inom hakparentes utgör endast en beskrivning av det ord som skulle ha stått där, men som har tagits bort av integritetsskäl.

Tabell 9: Viktigaste ord för modeller tränade på rå textdata.

Modell	Koppling till renovering	Ingen koppling till renovering
LR_TFIDF	(1) då, (2) kommer, (3) [adress], (4) höjas, (5) väl	(1) tror, (2) korttidskontrakt, (3) hörbart, (4) sambo, (5) flyttade
NB_TFIDF	(1) dan, (2) dagens, (3) fredag, (4) referensgruppen, (5) ökat	(1) studentlägenhet, (2) angavs, (3) energieffektiviseringen, (4) fönsterbytet, (5) korttidskontrakt
SVM_TFIDF	(1) då, (2) kommer, (3) [adress], (4) höjas, (5) familjebostäder	(1) tror, (2) hörbart, (3) liksom, (4) korttidskontrakt, (5) sambo
LR_BoW	(1) kvm, (2) renovera, (3) kan, (4) borde, (5) höjas	(1) tror, (2) stor, (3) visste, (4) bra, (5) betala
NB_BoW	(1) censurerat, (2) dan, (3) fb, (4) dethär, (5) mummel	(1) turkiska, (2) studentlägenhet, (3) avbryter, (4) fönsterbytet, (5) [stad]
SVM_BoW	(1) till en, (2) hyran, (3) sedan, (4) kan, (5) [adress]	(1) sambo, (2) bra, (3) flyttade, (4) korttidskontrakt, (5) stor

Tabell 10: Viktigaste ord för modeller tränade på stoppordsfiltrerad text.

Modell	Koppling till renovering	Ingen koppling till renovering
LR_TFIDF	(1) kommer, (2) [adress], (3) höjas, (4) väl, (5) renovera	(1) tror, (2) hörbart, (3) sambo, (4) korttidskontrakt, (5) [område]
NB_TFIDF	(1) fredag, (2) svartmögel, (3) östra, (4) ökat, (5) uppgången	(1) korttidskontrakt, (2) studentlägenhet, (3) angavs, (4) fönsterbytet, (5) energieffektiviseringen
SVM_TFIDF	(1) kommer, (2) [adress], (3) höjas, (4) väl, (5) [adress]	(1) tror, (2) hörbart, (3) sambo, (4) stor, (5) korttidskontrakt
LR_BoW	(1) renovera, (2) jobbigt, (3) höjas, (4) dyrare, (5) kvm	(1) sambo, (2) visste, (3) tror, (4) stor, (5) ohörbart
NB_BoW	(1) censurerat, (2) dan, (3) fb, (4) dethär, (5) mummel	(1) studentlägenhet, (2) avbryter, (3) [stad], (4) fönsterbytet, (5) [område]
SVM_BoW	(1) hyran, (2) [adress], (3) renovera, (4) dyrt, (5) också	(1) flyttat, (2) sambo, (3) genom, (4) ingenting, (5) korttidskontrakt

Tabell 11: Viktigaste ord för modeller tränade på lemmatiserad text.

Modell	Koppling till renovering	Ingen koppling till renovering
LR_TFIDF	(1) säga, (2) komma, (3) sen, (4) ja, (5) väl	(1) korttidskontrakt, (2) tro, (3) sambo, (4) hörbar, (5) liksom
NB_TFIDF	(1) dotta, (2) [adress], (3) fredag, (4) permanent, (5) besked	(1) korttidskontrakt, (2) studentlägenhet, (3) [område], (4) [område], (5) fönsterbyt
SVM_TFIDF	(1) säga, (2) evakueringslägenhet, (3) [adress], (4) komma, (5) skola renovera	(1) hörbar, (2) tro, (3) korttidskontrakt, (4) bra, (5) sambo
LR_BoW	(1) dyr, (2) skola renovera, (3) jobbig, (4) höja, (5) evakueringslägenhet	(1) bra, (2) tro, (3) sambo, (4) korttidskontrakt, (5) svar
NB_BoW	(1) dan, (2) räkning, (3) kläder, (4) mini, (5) dotta	(1) studentlägenhet, (2) fönsterbyt, (3) [område], (4) [stad], (5) hav
SVM_BoW	(1) dyr, (2) skola renovera, (3) spara, (4) kvm ja, (5) jobbig	(1) sambo, (2) genom, (3) bero, (4) korttidskontrakt, (5) ingenting

Tabell 12: Viktigaste ord för modeller tränade på synonymnormaliserad text.

Modell	Koppling till renovering	Ingen Koppling till renovering
LR_TFIDF	(1) säga, (2) komma, (3) väl, (4) sen, (5) [adress]	(1) korttidskontrakt, (2) tro, (3) hörbar, (4) sambo, (5) jättebra
NB_TFIDF	(1) två annan, (2) hjälpa flytt, (3) gammal område, (4) dotta, (5) göra riktig	(1) korttidskontrakt, (2) studentlägenhet, (3) flytt ingen, (4) fönsterbyt, (5) ingenting renovering
SVM_TFIDF	(1) säga, (2) komma, (3) väl, (4) sen, (5) [adress]	(1) tro, (2) hörbar, (3) korttidskontrakt, (4) sambo, (5) bra
LR_BoW	(1) höja, (2) evakueringslägenhet, (3) jobbig, (4) dyr, (5) skola renovera	(1) bra, (2) tro, (3) korttidskontrakt, (4) jättebra, (5) sambo
NB_BoW	(1) dan, (2) göra riktig, (3) gammal område, (4) två annan, (5) gå hyra	(1) studentlägenhet, (2) fönsterbyt, (3) [område], (4) [stad], (5) tänka betala
SVM_BoW	(1) dyr, (2) skola renovera, (3) jobbig, (4) evakueringslägenhet, (5) problem	(1) sambo, (2) renovering göra, (3) korttidskontrakt, (4) svar, (5) genom

J Källkod

J.1 Textextrahering och förbehandling

```
import os
from docx import Document
import re
import json
import nltk
from nltk.corpus import stopwords
import spacy

nlp = spacy.load("sv_core_news_sm")
folder_path = "files"

# Dessa två är tomma för att inte lämna ut namn på intervjuare
interviewerLabels = [
]
interviewerNames = [
]

skipStartswith = [
    "telefonintervju",
    "samtalets längd",
    "ljudfil",
    "tid",
    "intervju",
    "intervjuare",
    "längd",
    "intervjun",
    "inspelning",
    "respondent",
    *interviewerLabels,
    *interviewerNames,
]

skipTerms = [s.lower() for s in skipStartswith] + \
    [s.lower() for s in interviewerLabels] + \
    [s.lower() for s in interviewerNames]

# Regex patterns för att undvika och extrahera information
P_LINE = re.compile(r"~\s*P\d{1,3}:\b", re.IGNORECASE)

SKIP_START_PATTERN = re.compile(
    r"~\s*(?:"
    + "|".join(re.escape(s.lower()) for s in (skipStartswith + interviewerLabels + interviewerNames))
    + r")\b",
    re.IGNORECASE
)

PX_REMOVE_PATTERN = re.compile(r"~\s*P\s*d{1,3}\s*(?:[:|-|\\|.|\\]\s*|s+)", re.IGNORECASE)

WORD_PATTERN = re.compile(r"[a-zääö]+", re.IGNORECASE)

P_RESPONDENT_PATTERN = re.compile(
    r"~\s*P\d{1,3}\s*=\s*respondent\s*$",
    re.IGNORECASE
)

INTERVIEWER_PHRASES_REMOVE = re.compile(
    r"(?!~\w)(?:"
    + "|".join(re.escape(s.lower()) for s in sorted(interviewerLabels + interviewerNames, key=len, reverse=True))
    + r") (?!~\w)",
    re.IGNORECASE
)

INTERVIEWER_WORDS = set()
for lbl in interviewerLabels:
    for w in re.findall(r"[a-zääö]+", lbl.lower()):
        INTERVIEWER_WORDS.add(w)
for name in interviewerNames:
    for w in re.findall(r"[a-zääö]+", name.lower()):
        INTERVIEWER_WORDS.add(w)

# "Klassificerar" raden och bestämmer ifall den ska skippas eller inte
def classify_line(text: str) -> str:
    if not text or not text.strip():
        return "skip"

    if P_RESPONDENT_PATTERN.search(text):
        return "skip"

    if P_LINE.search(text):
        return "include"

    if SKIP_START_PATTERN.search(text):
```

```

        return "skip"

    return "naked"

swedish_stopwords = set(stopwords.words("swedish"))
pronouns_to_remove = {"han", "hon", "hen"}

words_to_remove = {
    "f", "m", "b", "då", "den",
    "dem", "ca", "är", "var", "där", "tror",
    "skulle", "ja", "du", "jag", "sa", "säger",
    "liksom", "alltid",
}

words_raw = {}
words_no_stopwords = {}
words_lemmatized = {}
words_semi_final = {}
#Går igenom alla filer och extraherar information
for filename in os.listdir(folder_path):
    if not filename.endswith(".docx"):
        continue
    file_path = os.path.join(folder_path, filename)
    print(f"Läser fil: {filename}")

    doc = Document(file_path)

    raw_tokens_all = []
    no_stopwords_tokens_all = []
    lemmatized_tokens_all = []
    semi_final_tokens_all = []
    #Extraherar ord och tar bort ord som inte ska vara med
    for paragraph in doc.paragraphs:
        text = paragraph.text.strip()
        verdict = classify_line(text)

        if verdict == "skip":
            continue

        cleaned = PX_REMOVE_PATTERN.sub("", text).lower()
        cleaned = INTERVIEWER_PHRASES_REMOVE.sub(" ", cleaned)

        tokens_raw = WORD_PATTERN.findall(cleaned)

        tokens_raw = [t for t in tokens_raw if t not in INTERVIEWER_WORDS]

        raw_tokens_all.extend(tokens_raw)

        tokens_no_stop = [t for t in tokens_raw if t not in pronouns_to_remove]
        tokens_no_stop = [t for t in tokens_no_stop if t not in swedish_stopwords]
        no_stopwords_tokens_all.extend(tokens_no_stop)

        doc = nlp(" ".join(tokens_no_stop))
        tokens_lemma = [token.lemma_.lower() for token in doc]

        lemmatized_tokens_all.extend(tokens_lemma)

        tokens_semi = [t for t in tokens_lemma if t not in words_to_remove]

        semi_final_tokens_all.extend(tokens_semi)

    key = os.path.splitext(filename)[0]

    words_raw[key] = raw_tokens_all
    words_no_stopwords[key] = no_stopwords_tokens_all
    words_lemmatized[key] = lemmatized_tokens_all
    words_semi_final[key] = semi_final_tokens_all

#Sparar alla ord till respektive json fil, semi_final används sedan för synonymnormalisering
output_folder = "json"
os.makedirs(output_folder, exist_ok=True)
with open(os.path.join(output_folder, "words_raw.json"), "w", encoding="utf-8") as f:
    json.dump(words_raw, f, ensure_ascii=False, indent=2)

with open(os.path.join(output_folder, "words_no_stopwords.json"), "w", encoding="utf-8") as f:
    json.dump(words_no_stopwords, f, ensure_ascii=False, indent=2)

with open(os.path.join(output_folder, "words_lemmatized.json"), "w", encoding="utf-8") as f:
    json.dump(words_lemmatized, f, ensure_ascii=False, indent=2)

with open(os.path.join(output_folder, "words_semi_final.json"), "w", encoding="utf-8") as f:
    json.dump(words_semi_final, f, ensure_ascii=False, indent=2)

```

J.1.1 Klustring och skapande av synonymmappning

```
import os
```



```

import json
from collections import Counter, defaultdict
from sentence_transformers import SentenceTransformer
from sklearn.cluster import AgglomerativeClustering

folder_path = "json"
input_file = os.path.join(folder_path, "words_semi_final.json")

clusters_out = os.path.join(folder_path, "auto_clusters.json")
syn_map_out = os.path.join(folder_path, "auto_synonym_map.json")

#Extraherar information från semi_final.json
with open(input_file, "r", encoding="utf-8") as f:
    wordsPerPerson = json.load(f)

#Samlar alla tokens från semi_final.json
all_tokens = []
for person, tokens in wordsPerPerson.items():
    all_tokens.extend(tokens)

#Räknar hur ofta varje ord förekommer
freq = Counter(all_tokens)
#Ord med en frekvens av 5 och högre kommer att användas och skapa kluster och identifiera potentiella synonymer
min_Freq = 5
vocab = [w for w, c in freq.items() if c >= min_Freq]

#Laddar in en förtränad språkmodell från sentence_transformer
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
#Ord omvandlas till numeriska vektorer
embeddings = model.encode(vocab, normalize_embeddings=True, show_progress_bar=True)
#Max distans tillåtet mellan ord i samma kluster är 0.225
dist_Threshold = 0.225

#Inställningar för den agglomerativa klustringen
cluster_model = AgglomerativeClustering(
    n_clusters=None,
    metric="euclidean",
    linkage="average",
    distance_threshold=dist_Threshold
)

#Klustrar orden
labels = cluster_model.fit_predict(embeddings)
clusters = defaultdict(list)
for word, label in zip(vocab, labels):
    clusters[int(label)].append(word)

#Skapa en synonym map
synonym_map = {}
cluster_info = []
for cluster_id, words in clusters.items():
    #Sorterar orden utifrån frekvens
    words_sorted = sorted(words, key=lambda w: freq[w], reverse=True)

    #Tar det mest vanliga ordet och sätter det till standard formen
    canonical = words_sorted[0]
    #Dem andra orden i klustret mappas till standardordet
    for w in words:
        if w != canonical:
            synonym_map[w] = canonical

    cluster_info.append({
        "cluster_id": cluster_id,
        "canonical": canonical,
        "words": words_sorted,
        "size": len(words)
    })

#Sorterar kluster
cluster_info.sort(key=lambda x: x["size"], reverse=True)

#Skapar en json fil med kluster
with open(clusters_out, "w", encoding="utf-8") as f:
    json.dump(cluster_info, f, ensure_ascii=False, indent=2)
#Skapar en synonym map json fil för senare transformering
with open(syn_map_out, "w", encoding="utf-8") as f:
    json.dump(synonym_map, f, ensure_ascii=False, indent=2)

```

J.1.2 Synonymvandling

```

import os
import json

folder_path = "json"

```

```

input_words_file = os.path.join(folder_path, "words_semi_final.json")
input_synonym_file = os.path.join(folder_path, "auto_synonym_map.json")
output_file = os.path.join(folder_path, "words_final.json")

#Laddar in ord från semi_final
with open(input_words_file, "r", encoding="utf-8") as f:
    words_per_person = json.load(f)
#Laddar in synonym map
with open(input_synonym_file, "r", encoding="utf-8") as f:
    synonym_map = json.load(f)

final_words_per_person = {}
#Byter ut ord mot deras synonymer enligt synonym map
for person_id, words in words_per_person.items():
    replaced_words = [synonym_map.get(word, word) for word in words]
    final_words_per_person[person_id] = replaced_words

#Sparar allt i en slutlig json fil words_final.json
with open(output_file, "w", encoding="utf-8") as f:
    json.dump(final_words_per_person, f, ensure_ascii=False, indent=2)

```

J.2 Modellernas kärnkod

Koden i detta appendix utgör ett urval av de centrala delarna av implementationen. Kod för visualisering, filhantering, export av resultat och andra stödjande funktioner har utelämnats för att tydliggöra de algoritmer och metoder som ligger till grund för studiens resultat.

J.2.1 Naiv Bayes

```

from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import StratifiedKFold
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from tune_utils import (
    evaluate_fold,
    passes_class_constraint,
    aggregate_metrics,
    balanced_sample,
    grid_rows,
    vec_params_to_string,
)

# Optimeringsmål och klasskrav
OPTIMIZE_FOR = "f1_macro_mean"
MIN_CLASS_ACC = 0.50 # Varje klass måste ha minst 50 % klassvis noggrannhet

BASE = Path(__file__).parent
PARENT = BASE.parent

# datafiler
TRAIN_CSVS = {
    "raw": PARENT / "train_raw.csv",
    "no_stopwords": PARENT / "train_no_stopwords.csv",
    "lemmatized": PARENT / "train_lemmatized.csv",
    "final": PARENT / "train_final.csv",
}

SPLITS = 5 # Antal folds i korsvalidering
RANDOM_STATE = 42 # Reproducerbarhet
DOCS_PER_CLASS = 93 # Antal dokument per klass vid balanserad sampling

# Stage 1: kombinationer av hyperparametrar testas för BoW och TFIDF
VEC_PARAM_GRIDS = {
    "BoW": {

```

```

        "min_df": [1, 2, 3, 4, 5],
        "max_df": [0.8, 0.9, 1.0],
        "ngram_range": [(1, 1), (1, 2)]
    },
    "TFIDF": {
        "min_df": [1, 2, 3, 4, 5],
        "max_df": [0.8, 0.9, 1.0],
        "ngram_range": [(1, 1), (1, 2)]
    },
}

# Stage 2: alpha söks i log-skala mellan 0.001 och 1000 (50 punkter)
NB_PARAM_GRID = {"alpha": np.logspace(-3, 3, 50)}

# Hjälpfunktioner

def evaluate_text_representations(X, y, alpha=0.1):
    """Utvärderar alla kombinationer av vektoriseringsparametrar med fast alpha=0.1.
    """
    class_labels = sorted(y.unique())
    kf = StratifiedKFold(n_splits=SPLITS, shuffle=True, random_state=RANDOM_STATE)

    all_results = []

    for vec_name, vec_grid in VEC_PARAM_GRIDS.items():
        param_combos = grid_rows(vec_grid)

        for vec_params in param_combos:
            metrics_list = []
            all_class_accs = []

            for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
                # Dela upp data i träning och validering för aktuell fold.
                X_train_full = X.iloc[train_idx].reset_index(drop=True)
                y_train_full = y.iloc[train_idx].reset_index(drop=True)
                X_val = X.iloc[val_idx].reset_index(drop=True)
                y_val = y.iloc[val_idx].reset_index(drop=True)

                # Balansera träningsdata
                X_train, y_train = balanced_sample(
                    X_train_full, y_train_full, DOCS_PER_CLASS, RANDOM_STATE + fold
                )

                vec = CountVectorizer(**vec_params) if vec_name == "BoW" else
                ↪ TfIdfVectorizer(**vec_params)
                clf = MultinomialNB(alpha=alpha)
                pipe = Pipeline([("vec", vec), ("clf", clf)])
                metrics, per_class_acc = evaluate_fold(pipe, X_train, y_train, X_val,
                ↪ y_val, class_labels)
                metrics_list.append(metrics)
                all_class_accs.append(per_class_acc)

            # Kontrollera att alla klasser uppfyller MIN_CLASS_ACC i varje fold
            constraint_passed = all((passes_class_constraint(acc, MIN_CLASS_ACC) for acc in
            ↪ all_class_accs))
            agg = aggregate_metrics(metrics_list)
            agg["vectorizer"] = vec_name
            agg["min_df"] = vec_params.get("min_df")
            agg["max_df"] = vec_params.get("max_df")
            agg["ngram_range"] = vec_params.get("ngram_range")
            agg["vectorizer_params"] = vec_params_to_string(vec_params)

```

```

        agg["passes_constraint"] = constraint_passed

    all_results.append(agg)

results_df = pd.DataFrame(all_results)
sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in results_df.columns else "f1_macro_mean"
return results_df.sort_values(sort_col, ascending=False)

def optimize_alpha_for_best_textrep(X, y, best_vec_name, best_vec_params):
    """Söker det bästa alpha-värdet med iterativ zoom kring bästa hittills funna alpha.

    Börjar med ett grovt log-skalat rutnät (NB_PARAM_GRID), zoomar sedan in
    kring det bästa alpha-värdet tills F1 inte förbättras mer än tol.
    """
    class_labels = sorted(y.unique())
    kf = StratifiedKFold(n_splits=SPLITS, shuffle=True, random_state=RANDOM_STATE)

    best_score = -np.inf
    tol = 1e-4 # Minsta förbättring för att fortsätta zooma
    zoom_step = 0
    current_alphas = list(NB_PARAM_GRID["alpha"])
    searched_alphas = set()
    improved = True
    all_results = []

    while improved and zoom_step < 5:
        improved = False
        results = []

        for alpha in current_alphas:
            metrics_list = []
            all_class_accs = []

            for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
                X_train_full = X.iloc[train_idx].reset_index(drop=True)
                y_train_full = y.iloc[train_idx].reset_index(drop=True)
                X_val = X.iloc[val_idx].reset_index(drop=True)
                y_val = y.iloc[val_idx].reset_index(drop=True)
                # Balansera träningsdata
                X_train, y_train = balanced_sample(
                    X_train_full, y_train_full, DOCS_PER_CLASS, RANDOM_STATE + fold
                )

                vec = CountVectorizer(**best_vec_params) if best_vec_name == "BoW" else
                ↪ TfIdfVectorizer(**best_vec_params)
                clf = MultinomialNB(alpha=alpha)
                pipe = Pipeline([("vec", vec), ("clf", clf)])
                metrics, per_class_acc = evaluate_fold(pipe, X_train, y_train, X_val,
                ↪ y_val, class_labels)
                metrics_list.append(metrics)
                all_class_accs.append(per_class_acc)

            constraint_passed = all(passes_class_constraint(acc, MIN_CLASS_ACC) for acc in
            ↪ all_class_accs)
            agg = aggregate_metrics(metrics_list)
            agg["alpha"] = alpha
            agg["vectorizer"] = best_vec_name
            agg["min_df"] = best_vec_params.get("min_df")
            agg["max_df"] = best_vec_params.get("max_df")
            agg["ngram_range"] = best_vec_params.get("ngram_range")

```

```

    agg["vectorizer_params"] = vec_params_to_string(best_vec_params)
    agg["passes_constraint"] = constraint_passed

    results.append(agg)
    searched_alphas.add(round(float(alpha), 8))

results_df = pd.DataFrame(results)
sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in results_df.columns else "f1_macro_mean"
all_results.append(results_df.sort_values(sort_col, ascending=False))

best_row = results_df.loc[results_df[sort_col].idxmax()]
best_this_score = best_row[sort_col]
best_alpha = best_row["alpha"]
if best_this_score > best_score + tol:
    best_score = best_this_score
    improved = True
else:
    break # Ingen tillräcklig förbättring - avbryt zoom

# Zooma in kring bästa alpha
step = max(0.05, 0.1 * float(best_alpha))
new_alphas = [best_alpha + i * step for i in range(-4, 5)]
current_alphas = [a for a in new_alphas if round(float(a), 8) not in
    ↳ searched_alphas and a > 0]
zoom_step += 1
if not current_alphas:
    break

if all_results:
    final_df = pd.concat(all_results, ignore_index=True)
    final_df = final_df.drop_duplicates(subset=["alpha", "vectorizer",
    ↳ "vectorizer_params"], keep="first")
    sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in final_df.columns else "f1_macro_mean"
    final_df = final_df.sort_values(sort_col, ascending=False)
else:
    final_df = pd.DataFrame()
return final_df

# STAGE 1: Optimera textrepresentationsparametrar för alla CSV-filer
stage1_results = {}
best_params_by_csv = {}

for csv_name, csv_path in TRAIN_CSVS.items():
    df = pd.read_csv(csv_path)
    X = df["text"].fillna("").astype(str)
    y = df["label"].astype(str)

    results_df = evaluate_text_representations(X, y, alpha=0.1)
    stage1_results[csv_name] = results_df

# Välj bästa hyperparameterkombination per textrepresentation

for csv_name, results_df in stage1_results.items():

    best_params_by_csv[csv_name] = {}

    for vec_name in ["BoW", "TFIDF"]:
        vec_df = results_df[results_df["vectorizer"] == vec_name]
        if vec_df.empty:
            continue

```

```

sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in vec_df.columns else "f1_macro_mean"

passed_df = vec_df[vec_df["passes_constraint"]]
if not passed_df.empty:
    # Välj bästa bland de som klarar klassvis accuracy-kravet
    best = passed_df.sort_values(sort_col, ascending=False).iloc[0]
else:
    best = vec_df.sort_values(sort_col, ascending=False).iloc[0]
    print("Ingen hyperparameterkombination uppfyller kraven på klassvis
    ↪ noggrannhet")

best_params_by_csv[csv_name][vec_name] = {
    "min_df": int(best["min_df"]),
    "max_df": best["max_df"],
    "ngram_range": best["ngram_range"],
}

# STAGE 2: Optimera alpha för varje textrepresentation
stage2_results = {}

for csv_name, csv_path in TRAIN_CSVS.items():
    df = pd.read_csv(csv_path)
    X = df["text"].fillna("").astype(str)
    y = df["label"].astype(str)

    all_vec_results = []
    for vec_name in ["BoW", "TFIDF"]:
        if csv_name not in best_params_by_csv or vec_name not in
        ↪ best_params_by_csv[csv_name]:
            continue
        bvp = best_params_by_csv[csv_name][vec_name]
        vec_df = optimize_alpha_for_best_textrep(X, y, vec_name, bvp)
        if not vec_df.empty:
            all_vec_results.append(vec_df)

    if all_vec_results:
        combined = pd.concat(all_vec_results, ignore_index=True)
        combined = combined.sort_values(["vectorizer", OPTIMIZE_FOR], ascending=[True,
        ↪ False])
        stage2_results[csv_name] = combined

# Välj bästa alpha: prioritera de som klarar constraint
best_models = {}
for csv_name, results_df in stage2_results.items():
    best_models[csv_name] = {}

    for vec_name in ["BoW", "TFIDF"]:
        vec_df = results_df[results_df["vectorizer"] == vec_name]

        if vec_df.empty:
            continue

        passed_df = vec_df[vec_df["passes_constraint"]]

        if not passed_df.empty:
            best = passed_df.sort_values(
                OPTIMIZE_FOR,
                ascending=False

```

```

        ).iloc[0]
    else:
        best = vec_df.sort_values(
            OPTIMIZE_FOR,
            ascending=False
        ).iloc[0]

    best_models[csv_name][vec_name] = best

```

J.2.2 Logistisk regression

```

from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from tune_utils import (
    evaluate_fold,
    passes_class_constraint,
    aggregate_metrics,
    balanced_sample,
    grid_rows,
    vec_params_to_string,
)

# Optimeringsmål och klasskrav
OPTIMIZE_FOR = "f1_macro_mean"
MIN_CLASS_ACC = 0.50

BASE = Path(__file__).parent
PARENT = BASE.parent

# datafiler
TRAIN_CSVS = {
    "raw": PARENT / "train_raw.csv",
    "no_stopwords": PARENT / "train_no_stopwords.csv",
    "lemmatized": PARENT / "train_lemmatized.csv",
    "final": PARENT / "train_final.csv",
}

# Gemensamma körinställningar
SPLITS = 5
RANDOM_STATE = 42
DOCS_PER_CLASS = 93

# Stage 2: C söks i log-skala
LR_PARAM_GRID = {"C": np.logspace(-3, 3, 50)}

# Stage 1: kombinationer av hyperparametrar testas för BoW och TFIDF
VEC_PARAM_GRIDS = {
    "BoW": {"min_df": [1, 2, 3, 4, 5], "max_df": [0.8, 0.9, 1.0], "ngram_range": [(1, 1),
    ↪ (1, 2)]},
    "TFIDF": {"min_df": [1, 2, 3, 4, 5], "max_df": [0.8, 0.9, 1.0], "ngram_range": [(1,
    ↪ 1), (1, 2)]},
}

# Stage 1: Optimera textrepresentationers hyperparametrar
def evaluate_text_representations(X, y, C=1.0):

```

```

"""Utvärderar alla kombinationer av vektoriseringsparametrar med fast C=1.0.
    """
results = []
kf = StratifiedKFold(n_splits=SPLITS, shuffle=True, random_state=RANDOM_STATE)
class_labels = sorted(y.unique())

for vec_name in ["BoW", "TFIDF"]:
    for vec_params in grid_rows(VEC_PARAM_GRIDS[vec_name]):
        metrics_list = []
        all_class_accs = []

        for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
            X_train_full = X.iloc[train_idx].reset_index(drop=True)
            y_train_full = y.iloc[train_idx].reset_index(drop=True)
            X_val = X.iloc[val_idx].reset_index(drop=True)
            y_val = y.iloc[val_idx].reset_index(drop=True)
            # Balansera träningsdata
            X_train, y_train = balanced_sample(
                X_train_full, y_train_full, DOCS_PER_CLASS, RANDOM_STATE + fold
            )

            vec = CountVectorizer(**vec_params) if vec_name == "BoW" else
                ↪ TfidfVectorizer(**vec_params)
            clf = LogisticRegression(C=C, max_iter=1000, random_state=RANDOM_STATE,
                ↪ solver='lbfgs')
            pipe = Pipeline([("vec", vec), ("clf", clf)])
            metrics, per_class_acc = evaluate_fold(pipe, X_train, y_train, X_val,
                ↪ y_val, class_labels)
            metrics_list.append(metrics)
            all_class_accs.append(per_class_acc)

            # Kontrollera att alla klasser uppfyller MIN_CLASS_ACC i varje fold
            constraint_passed = all(passes_class_constraint(acc, MIN_CLASS_ACC) for acc in
                ↪ all_class_accs)
            agg = aggregate_metrics(metrics_list)
            agg["vectorizer"] = vec_name
            agg["min_df"] = vec_params.get("min_df")
            agg["max_df"] = vec_params.get("max_df")
            agg["ngram_range"] = vec_params.get("ngram_range")
            agg["vectorizer_params"] = vec_params_to_string(vec_params)
            agg["passes_constraint"] = constraint_passed

            results.append(agg)

return pd.DataFrame(results)

# Stage 2: Optimera C
def optimize_C_for_best_textrep(X, y, best_vec_name, best_vec_params):
    """Stage 2: Optimera C med bästa textrepresentation."""
    init-Cs = list(LR_PARAM_GRID["C"])
    all_results = []
    searched-Cs = set()
    best_score = -np.inf
    improved = True
    current-Cs = init-Cs
    tol = 1e-4 # Minsta förbättring för att fortsätta zooma
    max_zoom_steps = 10
    zoom_step = 0
    kf = StratifiedKFold(n_splits=SPLITS, shuffle=True, random_state=RANDOM_STATE)
    class_labels = sorted(y.unique())

```



```

while improved and zoom_step < max_zoom_steps:
    results = []
    for C in current-Cs:
        metrics_list = []
        all_class_accs = []

        for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
            X_train_full = X.iloc[train_idx].reset_index(drop=True)
            y_train_full = y.iloc[train_idx].reset_index(drop=True)
            X_val = X.iloc[val_idx].reset_index(drop=True)
            y_val = y.iloc[val_idx].reset_index(drop=True)
            X_train, y_train = balanced_sample(
                X_train_full, y_train_full, DOCS_PER_CLASS, RANDOM_STATE + fold
            )

            vec = CountVectorizer(**best_vec_params) if best_vec_name == "BoW" else
            ↪ TfidfVectorizer(**best_vec_params)
            clf = LogisticRegression(C=C, max_iter=1000, random_state=RANDOM_STATE,
            ↪ solver='lbfgs')
            pipe = Pipeline([("vec", vec), ("clf", clf)])
            metrics, per_class_acc = evaluate_fold(pipe, X_train, y_train, X_val,
            ↪ y_val, class_labels)
            metrics_list.append(metrics)
            all_class_accs.append(per_class_acc)

        constraint_passed = all((passes_class_constraint(acc, MIN_CLASS_ACC) for acc in
        ↪ all_class_accs))
        agg = aggregate_metrics(metrics_list)
        agg["C"] = C
        agg["vectorizer"] = best_vec_name
        agg["min_df"] = best_vec_params.get("min_df")
        agg["max_df"] = best_vec_params.get("max_df")
        agg["ngram_range"] = best_vec_params.get("ngram_range")
        agg["vectorizer_params"] = vec_params_to_string(best_vec_params)
        agg["passes_constraint"] = constraint_passed
        agg["is_initial_search"] = (zoom_step == 0)

        results.append(agg)
        searched-Cs.add(round(float(C), 8))

results_df = pd.DataFrame(results)
sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in results_df.columns else "f1_macro_mean"
all_results.append(results_df.sort_values(sort_col, ascending=False))

best_row = results_df.loc[results_df[sort_col].idxmax()]
best_this_score = best_row[sort_col]
best_C = best_row["C"]
if best_this_score > best_score + tol:
    best_score = best_this_score
    improved = True
else:
    break

# Zooma in kring bästa C
step = max(0.05, 0.1 * float(best_C))
new-Cs = [best_C + i * step for i in range(-4, 5)]
current-Cs = [c for c in new-Cs if round(float(c), 8) not in searched-Cs and c >
↪ 0]
zoom_step += 1
if not current-Cs:
    break

```

```

if all_results:
    final_df = pd.concat(all_results, ignore_index=True)
    final_df = final_df.drop_duplicates(subset=["C", "vectorizer",
        ↪ "vectorizer_params"], keep="first")
    sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in final_df.columns else "f1_macro_mean"
    final_df = final_df.sort_values(sort_col, ascending=False)
else:
    final_df = pd.DataFrame()
return final_df

# STAGE 1: Optimera textrepresentationsparametrar för alla CSV-filer
stage1_results = {}
best_params_by_csv = {}

for csv_name, csv_path in TRAIN_CSVS.items():
    df = pd.read_csv(csv_path)
    X = df["text"].fillna("").astype(str)
    y = df["label"].astype(str)

    results_df = evaluate_text_representations(X, y, C=1.0)
    stage1_results[csv_name] = results_df

# Välj bästa hyperparameterkombination per textrepresentation
for csv_name, results_df in stage1_results.items():

    best_params_by_csv[csv_name] = {}

    for vec_name in ["BoW", "TFIDF"]:
        vec_df = results_df[results_df["vectorizer"] == vec_name]
        if vec_df.empty:
            continue
        sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in vec_df.columns else "f1_macro_mean"

        if not passed_df.empty:
            # Välj bästa bland de som klarar klassvis accuracy-kravet
            best = passed_df.sort_values(sort_col, ascending=False).iloc[0]
        else:
            best = vec_df.sort_values(sort_col, ascending=False).iloc[0]
            print("Ingen hyperparameterkombination uppfyller kraven på klassvis
                ↪ noggrannhet")

        best_params_by_csv[csv_name][vec_name] = {
            "min_df": int(best["min_df"]),
            "max_df": best["max_df"],
            "ngram_range": best["ngram_range"],
        }

# STAGE 2: Optimera C för varje textrepresentation
stage2_results = {}

for csv_name, csv_path in TRAIN_CSVS.items():
    df = pd.read_csv(csv_path)
    X = df["text"].fillna("").astype(str)
    y = df["label"].astype(str)

    all_vec_results = []
    for vec_name in ["BoW", "TFIDF"]:

```

```

if csv_name not in best_params_by_csv or vec_name not in
↳ best_params_by_csv[csv_name]:
    continue
bvp = best_params_by_csv[csv_name][vec_name]
vec_df = optimize_C_for_best_textrep(X, y, vec_name, bvp)
if not vec_df.empty:
    all_vec_results.append(vec_df)

if all_vec_results:
    combined = pd.concat(all_vec_results, ignore_index=True)
    combined = combined.sort_values(["vectorizer", OPTIMIZE_FOR], ascending=[True,
↳ False])
    stage2_results[csv_name] = combined

# Välj bästa C
for csv_name, results_df in stage2_results.items():
    for vec_name in ["BoW", "TFIDF"]:
        vec_df = results_df[results_df["vectorizer"] == vec_name]
        if vec_df.empty:
            continue

        passed_df = vec_df[vec_df["passes_constraint"]]
        if not passed_df.empty:
            best = passed_df.sort_values(OPTIMIZE_FOR, ascending=False).iloc[0]
        else:
            best = vec_df.sort_values(OPTIMIZE_FOR, ascending=False).iloc[0]
            print("Ingen hyperparameterkombination uppfyller kraven på klassvis
↳ noggrannhet")

```

J.2.3 SVM

```

from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import StratifiedKFold
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from tune_utils import (
    evaluate_fold,
    passes_class_constraint,
    aggregate_metrics,
    balanced_sample,
    grid_rows,
    vec_params_to_string,
)

# Optimeringsmål och klasskrav
OPTIMIZE_FOR = "f1_macro_mean"
MIN_CLASS_ACC = 0.50 # Varje klass måste ha minst 50 % klassvis noggrannhet

BASE = Path(__file__).parent
PARENT = BASE.parent

# datafiler
TRAIN_CSVS = {
    "raw": PARENT / "train_raw.csv",

```

```

    "no_stopwords": PARENT / "train_no_stopwords.csv",
    "lemmatized": PARENT / "train_lemmatized.csv",
    "final": PARENT / "train_final.csv",
}

SPLITS = 5          # Antal folds i korsvalidering
RANDOM_STATE = 42    # Reproducerbarhet
DOCS_PER_CLASS = 93 # Antal dokument per klass vid balanserad sampling

# Stage 2: C söks i log-skala mellan 0.001 och 1000 (30 punkter)
SVM_PARAM_GRID = {"C": np.logspace(-3, 3, 30)}

# Stage 1: kombinationer av hyperparametrar testas för BoW och TFIDF
VEC_PARAM_GRIDS = {
    "BoW": {"min_df": [1, 2, 3, 4, 5], "max_df": [0.8, 0.9, 1.0], "ngram_range": [(1,
↪ 1), (1, 2)]},
    "TFIDF": {"min_df": [1, 2, 3, 4, 5], "max_df": [0.8, 0.9, 1.0], "ngram_range": [(1,
↪ 1), (1, 2)]},
}

# Hjälpfunktioner
def make_svm_pipeline(vec_name, vec_params, C):
    """Bygger en sklearn-pipeline: vektoriserare + LinearSVC med givet C."""
    vec = CountVectorizer(**vec_params) if vec_name == "BoW" else
↪ TfidfVectorizer(**vec_params)
    clf = LinearSVC(C=C, max_iter=10000, random_state=RANDOM_STATE)
    return Pipeline([("vec", vec), ("clf", clf)])

# Stage 1: Optimerar textrepresentationers hyperparametrar
def evaluate_text_representations(X, y, C=1.0):
    """Utvärderar alla kombinationer av vektoriseringsparametrar med fast C=1.0.
    """
    results = []
    kf = StratifiedKFold(n_splits=SPLITS, shuffle=True, random_state=RANDOM_STATE)
    class_labels = sorted(y.unique())

    for vec_name in ["BoW", "TFIDF"]:
        for vec_params in grid_rows(VEC_PARAM_GRIDS[vec_name]):
            metrics_list = []
            all_class_accs = []

            for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
                X_train_full = X.iloc[train_idx].reset_index(drop=True)
                y_train_full = y.iloc[train_idx].reset_index(drop=True)
                X_val = X.iloc[val_idx].reset_index(drop=True)
                y_val = y.iloc[val_idx].reset_index(drop=True)

                # Balansera träningsdata
                X_train, y_train = balanced_sample(
                    X_train_full, y_train_full, DOCS_PER_CLASS, RANDOM_STATE + fold
                )

                pipe = make_svm_pipeline(vec_name, vec_params, C)
                metrics, per_class_acc = evaluate_fold(pipe, X_train, y_train, X_val,
↪ y_val, class_labels)
                metrics_list.append(metrics)
                all_class_accs.append(per_class_acc)

    # Kontrollera att alla klasser uppfyller MIN_CLASS_ACC i varje fold

```

```

        constraint_passed = all(passes_class_constraint(acc, MIN_CLASS_ACC) for acc in
        ↪ all_class_accs)
        agg = aggregate_metrics(metrics_list)
        agg["vectorizer"] = vec_name
        agg["min_df"] = vec_params.get("min_df")
        agg["max_df"] = vec_params.get("max_df")
        agg["ngram_range"] = vec_params.get("ngram_range")
        agg["vectorizer_params"] = vec_params_to_string(vec_params)
        agg["passes_constraint"] = constraint_passed
        results.append(agg)

    return pd.DataFrame(results)

# Stage 2: Optimera C

def optimize_C_for_best_textrep(X, y, best_vec_name, best_vec_params):
    """Söker det bästa C-värdet med iterativ zoom kring bästa hittills funna C.

    Börjar med ett grovt log-skalat rutnät (SVM_PARAM_GRID), zoomar sedan in
    kring det bästa C-värdet tills F1 inte förbättras mer än tol.
    """
    init-Cs = list(SVM_PARAM_GRID["C"])
    all_results = []
    searched-Cs = set()
    best_score = -np.inf
    improved = True
    current-Cs = init-Cs
    tol = 1e-4 # Minsta förbättring för att fortsätta zooma
    max_zoom_steps = 10
    zoom_step = 0
    kf = StratifiedKFold(n_splits=SPLITS, shuffle=True, random_state=RANDOM_STATE)
    class_labels = sorted(y.unique())

    while improved and zoom_step < max_zoom_steps:
        results = []
        for C in current-Cs:
            metrics_list = []
            all_class_accs = []

            for fold, (train_idx, val_idx) in enumerate(kf.split(X, y), 1):
                X_train_full = X.iloc[train_idx].reset_index(drop=True)
                y_train_full = y.iloc[train_idx].reset_index(drop=True)
                X_val = X.iloc[val_idx].reset_index(drop=True)
                y_val = y.iloc[val_idx].reset_index(drop=True)
                X_train, y_train = balanced_sample(
                    X_train_full, y_train_full, DOCS_PER_CLASS, RANDOM_STATE + fold
                )

                pipe = make_svm_pipeline(best_vec_name, best_vec_params, C)
                metrics, per_class_acc = evaluate_fold(pipe, X_train, y_train, X_val,
                    ↪ y_val, class_labels)
                metrics_list.append(metrics)
                all_class_accs.append(per_class_acc)

            constraint_passed = all(passes_class_constraint(acc, MIN_CLASS_ACC) for acc in
            ↪ all_class_accs)
            agg = aggregate_metrics(metrics_list)
            agg["C"] = C
            agg["vectorizer"] = best_vec_name
            agg["min_df"] = best_vec_params.get("min_df")
            agg["max_df"] = best_vec_params.get("max_df")

```

```

        agg["ngram_range"] = best_vec_params.get("ngram_range")
        agg["vectorizer_params"] = vec_params_to_string(best_vec_params)
        agg["passes_constraint"] = constraint_passed
        agg["is_initial_search"] = (zoom_step == 0)
        results.append(agg)
        searched-Cs.add(round(float(C), 8))

    results_df = pd.DataFrame(results)
    sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in results_df.columns else "f1_macro_mean"
    all_results.append(results_df.sort_values(sort_col, ascending=False))

    best_row = results_df.loc[results_df[sort_col].idxmax()]
    best_this_score = best_row[sort_col]
    best_C = best_row["C"]

    if best_this_score > best_score + tol:
        best_score = best_this_score
        improved = True
    else:
        break # Ingen tillräcklig förbättring - avbryt zoom

    # Zooma in kring bästa C
    step = max(0.05, 0.1 * float(best_C))
    new-Cs = [best_C + i * step for i in range(-4, 5)]
    current-Cs = [c for c in new-Cs if round(float(c), 8) not in searched-Cs and c >
        ↪ 0]
    zoom_step += 1
    if not current-Cs:
        break

if all_results:
    final_df = pd.concat(all_results, ignore_index=True)
    final_df = final_df.drop_duplicates(subset=["C", "vectorizer",
        ↪ "vectorizer_params"], keep="first")
    sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in final_df.columns else "f1_macro_mean"
    final_df = final_df.sort_values(sort_col, ascending=False)
else:
    final_df = pd.DataFrame()
return final_df

# STAGE 1: Optimera textrepresentationsparametrar för alla CSV-filer

stage1_results = {}
best_params_by_csv = {}

for csv_name, csv_path in TRAIN_CSVS.items():
    df = pd.read_csv(csv_path)
    X = df["text"].fillna("").astype(str)
    y = df["label"].astype(str)

    results_df = evaluate_text_representations(X, y, C=1.0)
    stage1_results[csv_name] = results_df

# Välj bästa hyperparameterkombination per textrepresentation
for csv_name, results_df in stage1_results.items():
    best_params_by_csv[csv_name] = {}

    for vec_name in ["BoW", "TFIDF"]:
        vec_df = results_df[results_df["vectorizer"] == vec_name]

```

```

    if vec_df.empty:
        continue
    sort_col = OPTIMIZE_FOR if OPTIMIZE_FOR in vec_df.columns else "f1_macro_mean"

    passed_df = vec_df[vec_df["passes_constraint"]]
    if not passed_df.empty:
        # Välj bästa bland de som klarar klassvis accuracy-kravet
        best = passed_df.sort_values(sort_col, ascending=False).iloc[0]
    else:
        best = vec_df.sort_values(sort_col, ascending=False).iloc[0]
    print("Inga hyperparameterkombinationer uppfyller kraven på klassvis
    ↪ noggrannhet")

    best_params_by_csv[csv_name][vec_name] = {
        "min_df": int(best["min_df"]),
        "max_df": best["max_df"],
        "ngram_range": best["ngram_range"],
    }

# STAGE 2: Optimera C för varje textrepresentation
stage2_results = {}

for csv_name, csv_path in TRAIN_CSVS.items():
    df = pd.read_csv(csv_path)
    X = df["text"].fillna("").astype(str)
    y = df["label"].astype(str)

    all_vec_results = []
    for vec_name in ["BoW", "TFIDF"]:
        if csv_name not in best_params_by_csv or vec_name not in
        ↪ best_params_by_csv[csv_name]:
            continue
        bvp = best_params_by_csv[csv_name][vec_name]
        vec_df = optimize_C_for_best_textrep(X, y, vec_name, bvp)
        if not vec_df.empty:
            all_vec_results.append(vec_df)

    if all_vec_results:
        combined = pd.concat(all_vec_results, ignore_index=True)
        combined = combined.sort_values(["vectorizer", OPTIMIZE_FOR], ascending=[True,
        ↪ False])
        stage2_results[csv_name] = combined

# Välj bästa C
best_models = {}

for csv_name, results_df in stage2_results.items():
    best_models[csv_name] = {}

    for vec_name in ["BoW", "TFIDF"]:
        vec_df = results_df[results_df["vectorizer"] == vec_name]

        if vec_df.empty:
            continue

        passed_df = vec_df[vec_df["passes_constraint"]]

        if not passed_df.empty:
            best = passed_df.sort_values(
                OPTIMIZE_FOR,
                ascending=False

```

```

        ).iloc[0]
    else:
        best = vec_df.sort_values(
            OPTIMIZE_FOR,
            ascending=False
        ).iloc[0]

    best_models[csv_name][vec_name] = best

```

J.2.4 Gemensamma hjälpfunktioner

```

"""
tune_utils.py
=====
Gemensamma funktioner för modellträning
"""

import numpy as np
import pandas as pd
from itertools import product
from sklearn.metrics import accuracy_score, f1_score, recall_score, precision_score

def vec_params_to_string(vec_params):
    """Formatera textrepresentationers hyperparametrar till en sträng."""
    ngram = vec_params.get("ngram_range", (1, 1))
    return f"min_df={vec_params.get('min_df')}, max_df={vec_params.get('max_df')},  

    ↪ ngram={ngram[0]}-{ngram[1]}"

def grid_rows(grid):
    """Bygg en lista med alla parameterkombinationer (kartesisk produkt)."""
    keys = list(grid.keys())
    values = [grid[k] for k in keys]
    return [dict(zip(keys, combo)) for combo in product(*values)]

def balanced_sample(X_s, y_s, n, rng_state):
    """Sampla exakt n dokument per klass."""
    X_s = pd.Series(X_s).reset_index(drop=True)
    y_s = pd.Series(y_s).reset_index(drop=True).astype(str)
    idx = (
        pd.DataFrame({"y": y_s})
        .groupby("y", group_keys=False)
        .apply(lambda g: g.sample(n=n, random_state=rng_state), include_groups=False)
        .index
    )
    return X_s.iloc[idx].reset_index(drop=True), y_s.iloc[idx].reset_index(drop=True)

def evaluate_fold(pipe, X_train, y_train, X_val, y_val, class_labels):
    """Träna och utvärdera pipeline på en fold, returnera metric och  

    ↪ per-klass-accuracy."""
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_val)
    metrics = {
        "accuracy": accuracy_score(y_val, pred),
        "f1_weighted": f1_score(y_val, pred, average="weighted"),
        "f1_macro": f1_score(y_val, pred, average="macro"),
        "recall_weighted": recall_score(y_val, pred, average="weighted"),
        "precision_weighted": precision_score(y_val, pred, average="weighted",
        ↪ zero_division=0),
    }
    # klassvis noggrannhet
    per_class_acc = {}
    for label in class_labels:

```



```

        mask = (y_val == label)
        if mask.sum() > 0:
            per_class_acc[label] = (pred[mask] == y_val[mask]).mean()
        else:
            per_class_acc[label] = np.nan
    return metrics, per_class_acc

def passes_class_constraint(per_class_accs, min_acc):
    """Returnera True om båda klasser uppfyller minimum krav på klassvis noggrannhet """
    for acc in per_class_accs.values():
        if not np.isnan(acc) and acc < min_acc:
            return False
    return True

def aggregate_metrics(metrics_list):
    """Beräkna medelvärde och standardavvikelse för varje mått över samtliga folds."""
    agg = {}
    for key in metrics_list[0].keys():
        vals = [m[key] for m in metrics_list]
        agg[f"{key}_mean"] = np.mean(vals)
        agg[f"{key}_std"] = np.std(vals)
    return agg

```